

Dynamisch Docker-omgevingbeheer voor CTF-events op on-prem infrastructuur.

Geïsoleerde challenge-omgevingen per speler of team.

Manu Devloo.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. T. Clauwaert

Co-promotor: Dhr. L. Singier

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Deze bachelorproef werd uitgevoerd in het kader van de opleiding Professionele bachelor toegepaste informatica aan HOGENT. Mijn interesse in cybersecurity en softwareontwikkeling vormde de aanleiding om dit onderwerp te kiezen. Binnen dit onderzoek werd gezocht naar een aanpak die veilig is en praktisch inzetbaar blijft op eigen hardware.

Graag bedank ik mijn promotor, dhr. Thomas Clauwaert, en mijn co-promotor, dhr. Laurens Singier, voor hun begeleiding, feedback en kritische vragen tijdens dit traject.

Samenvatting

Capture The Flag (CTF)-events zijn cybersecuritywedstrijden waarbij deelnemers technische uitdagingen oplossen om punten te behalen. Een deel van die uitdagingen vereist een netwerktoegankelijke service, bijvoorbeeld een webapplicatie of kwetsbare containeromgeving. Op on-premise infrastructuur worden dergelijke challenge-omgevingen vaak nog manueel of met losse scripts beheerd. Dat verhoogt de operationele belasting voor organisatoren en vergroot het risico op misconfiguraties, onvoldoende isolatie tussen deelnemers en resource-uitputting op de host.

Deze bachelorproef onderzoekt hoe Docker-containers die gekoppeld zijn aan CTFd-challenges dynamisch kunnen worden beheerd, aangemaakt en beëindigd op basis van gebruikersacties en configureerbare parameters binnen een volledig self-hosted context. De centrale onderzoeksvraag luidt hoe dit veilig, reproduceerbaar en schaalbaar kan gebeuren zonder afhankelijkheid van betaalde cloudcomponenten of Kubernetes als vereiste.

De methodologie combineert een literatuurstudie met praktijkgericht en iteratief onderzoek. Eerst werden CTFd, Docker, de Docker Engine API en direct toepasbare containerbeveiligingsmaatregelen bestudeerd. Daarna werd een verkennende standalone proof-of-concept gebouwd om container lifecycle management, timeout cleanup, netwerkisolatie, resource-limieten en idempotent startgedrag los van CTFd te valideren. Op basis van die bevindingen werd vervolgens een eigen CTFd-plugin ontwikkeld die een nieuw challenge-type toevoegt en de containerlevenscyclus rechtstreeks koppelt aan acties van spelers en beheerders.

De uiteindelijke plugin-PoC ondersteunt challengeconfiguratie met image, een of meerdere containerpoorten, CPU- en geheugenlimieten, timeout en een maximum aantal gelijktijdige instanties. Daarnaast ondersteunt de plugin het uploaden van Docker-archieven, het automatisch herimporteren van die archieven wanneer een image lokaal ontbreekt, het hergebruiken van een bestaande instantie voor dezelfde speler of hetzelfde team, automatische cleanup bij solve of timeout en een administratieve runtime-pagina binnen CTFd. Voor elke runtime worden basismaatregelen toegepast zoals een read-only root filesystem, capability dropping, no-new-privileges, PID-limieten, CPU- en geheugenlimieten en een afzonderlijk bridge-netwerk per instantie.

De validatie gebeurde in meerdere stappen. De standalone PoC werd gevalideerd met pytest, een Docker-gebaseerde smoke test, isolatie-experimenten, timeoutcontrole en een load snapshot met Locust. De plugin-PoC werd daarna

end-to-end getest in same-host modus, in een lokale split-host simulatie en in een reële opstelling met twee afzonderlijke Lima-VM's. Die validatie toont aan dat een CTFd-plugin zowel een lokale Docker-daemon als een remote Docker-host kan aansturen, terwijl spelers correcte runtime-URL's ontvangen voor de afzonderlijke challenge-host.

De conclusie van dit onderzoek is dat een plugin-gebaseerde aanpak binnen CTFd een haalbare en bruikbare oplossingsrichting vormt voor dynamisch containerbeheer bij CTF-events op eigen infrastructuur. Tegelijk blijkt dat verdere productierijpe uitwerking nog aandacht vraagt voor TLS-beveiligde remote Docker-communicatie, uitgebreidere foutinjectie, reverse-proxy-gebaseerde routing en schaalverdeling over meerdere runtime-hosts.

Inhoudsopgave

Lijst van figuren	ix
Lijst van tabellen	x
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	2
1.4 Scope en afbakening	3
1.5 Opzet van deze bachelorproef	3
2 Achtergrond en begrippenkader	4
2.1 Capture The Flag (CTF)	4
2.1.1 Types challenges	4
2.1.2 Deployment van challenges: statisch vs. dynamisch	5
2.1.3 Bekende CTF-platformen	5
2.2 CTFd als platform	5
2.2.1 Architectuur van CTFd	5
2.2.2 Het plugin-systeem	6
2.2.3 Challenge-beheer en granulariteit	6
2.3 Docker en containerisatie	6
2.3.1 Wat is Docker?	6
2.3.2 Docker Engine API	6
2.3.3 Docker SDK voor Python	7
2.3.4 Docker Compose	7
2.3.5 Resource management	7
2.3.6 Netwerkisolatie	7
2.4 Beveiliging van containers	7
2.4.1 Beveiligingsrisico's bij dynamisch aanmaken van containers	7
2.4.2 Direct toepasbare basismaatregelen	8
2.4.3 Container escape	8
2.4.4 Resource exhaustion en DoS-risico's	8
2.5 Load balancing zonder Kubernetes	8
2.5.1 Waarom geen Kubernetes?	8
2.5.2 Alternatieven	9
2.5.3 Wanneer is schaling nodig?	9

3	Methodologie	10
4	Huidige situatie	11
4.1	Organisatie van CTF-events op eigen infrastructuur	11
4.2	Manuele container deployments	11
4.3	Overzicht van bestaande tools en hun tekortkomingen	12
4.4	Conclusie: wat ontbreekt er?	12
5	Gewenste situatie	14
5.1	Vereisten voor de oplossing	14
5.1.1	Functionele vereisten	14
5.1.2	Niet-functionele vereisten	15
5.2	Architectuurvisie	15
5.2.1	Componenten en verantwoordelijkheden	15
5.2.2	Datastromen op hoofdlijnen	16
5.3	Granulariteitsbeslissing: container per gebruiker of per team	16
5.4	Schaalbaarheidsvereisten	17
6	Proof-of-Concept	19
6.1	Architectuuroverzicht	19
6.2	Verkennde standalone PoC	20
6.3	Implementatie van de CTFd-plugin	21
6.3.1	Custom challenge-type en configuratie	21
6.3.2	Start- en stopflow voor deelnemers	22
6.3.3	Timeout cleanup en cleanup bij solve	22
6.3.4	Administratieve runtime-interface	23
6.4	Beveiliging in de plugin-PoC	24
6.5	Deployment- en schaalbaarheidsoverwegingen	26
7	Validatie en testing	27
7.1	Testplan	27
7.1.1	Doelstellingen	27
7.1.2	Afbakening	27
7.2	Pre-validatie via de standalone PoC	28
7.2.1	Geautomatiseerde tests en API-validatie	28
7.2.2	End-to-end smoke test	28
7.2.3	Timeout, isolatie en resourcebeperking	28
7.2.4	Load snapshot met Locust	29
7.3	Validatie van de CTFd-plugin	29
7.3.1	Functionele end-to-end validatie	29
7.3.2	Split-host validatie	30
7.3.3	Reële validatie op twee VM's	31
7.3.4	Wat de pluginvalidatie laat zien	31

7.3.5	Wat bewust niet als bewezen claim wordt opgenomen	32
7.4	Interpretatie van de resultaten	32
8	Conclusie	33
8.1	Beantwoording van de onderzoeksvragen	33
8.2	Beperkingen van de PoC	34
8.3	Aanbevelingen voor productiegebruik	34
8.4	Mogelijke uitbreidingen	35
A	Onderzoeksvoorstel	36
A.1	Introductie	36
A.1.1	Probleemstelling	37
A.1.2	Onderzoeksvraag	37
A.1.3	Onderzoeksdoelstelling	38
A.2	State-of-the-art	38
A.3	Methodologie	38
A.4	Verwacht resultaat, conclusie	39
B	Installatie- en beheerinstructies	40
C	Configuratiebestanden	42
D	Testresultaten in detail	44
E	Onderzoekslog	47
	Bibliografie	50

Lijst van figuren

2.1	Architectuur van CTFd	6
5.1	Split-host architectuur	17
6.1	Architectuur van de plugin-PoC	20
6.2	Start-, stop- en cleanupflow	23

Lijst van tabellen

5.1	Vergelijking tussen container per gebruiker en per team	16
6.1	Vergelijking van beide PoC's	21

1

Inleiding

Capture The Flag (CTF)-events zijn cybersecuritywedstrijden waarbij deelnemers technische uitdagingen oplossen om punten te scoren (Innovirtuoso, [z.d.](#)). Ze worden georganiseerd door hogescholen, universiteiten, bedrijven en onafhankelijke gemeenschappen als educatieve en competitieve oefeningen. Elke deelnemer of elk team werkt aan zogenaamde *challenges*: geïsoleerde omgevingen met een kwetsbaarheid die uitgebuit moet worden om een *flag* te bemachtigen.

Naarmate CTF-events populairder worden, groeit ook de behoefte aan infrastructuur die dergelijke challenges betrouwbaar kan aanbieden. Op eigen infrastructuur worden challenges vandaag vaak manueel of semi-automatisch beheerd. Dat leidt tot operationele overhead, beveiligingsrisico's en schaalbaarheidsknelpunten (Singier, [2025](#)).

1.1. Probleemstelling

Bij het organiseren van CTF-events op eigen (on-premise) infrastructuur is het dynamisch en veilig beheren van containers voor challenges een bijzonder complexe opgave. Organisatoren moeten voor elke challenge een geïsoleerde containeromgeving opzetten, beheren en weer afbreken op basis van gebruikersacties. Dit proces verloopt vandaag grotendeels manueel of met eenvoudige scripts, wat de volgende problemen met zich meebrengt:

- Manuele interventie bij het opstarten en stopzetten van containers kost tijd en is foutgevoelig.
- Er is geen gegarandeerde isolatie tussen containers van verschillende teams of gebruikers.
- Resource-uitputting (CPU, RAM) door ongecontroleerde containers vormt een risico voor de stabiliteit van de host.

- Bestaande cloud-gebaseerde platformen (bijv. gehoste CTFd-diensten) zijn niet altijd geschikt voor organisaties die werken met privédata of op eigen hardware willen draaien.

De doelgroep van dit onderzoek zijn CTF-organisatoren en systeembeheerders die events organiseren op eigen on-premise infrastructuur en op zoek zijn naar een geautomatiseerde, veilige en schaalbare oplossing.

1.2. Onderzoeksvraag

De centrale onderzoeksvraag luidt:

Hoe kunnen Docker-containers die gelinkt zijn aan CTFd-challenges dynamisch worden beheerd, aangemaakt en beëindigd op basis van gebruikersacties en ingestelde parameters, op on-premise infrastructuur?

Hieruit volgen de volgende deelvragen:

- Welke beveiligingsmaatregelen zijn minimaal vereist om container-escapes en misbruik te voorkomen?
- Op welke manier kunnen resource-limieten (CPU, RAM) en netwerkisolatie technisch worden afgedwongen?
- Hoe kan een CTFd-plugin de Docker Engine API aanspreken zonder zware externe afhankelijkheden?
- Hoe valideert men de stabiliteit en veiligheid van de oplossing onder zware belasting?

1.3. Onderzoeksdoelstelling

Het beoogde resultaat van deze bachelorproef is een werkende proof-of-concept (PoC) die onderzoekt hoe CTFd-challenges dynamisch en veilig beheerd kunnen worden op eigen infrastructuur. De uiteindelijke PoC is een CTFd-plugin die communiceert met de Docker Engine API, resource-limieten afdwingt en netwerkisolatie toepast. Tijdens de uitwerking werd eerst een beperkte standalone proefopstelling gebouwd om de haalbaarheid van container lifecycle management, basisbeveiliging en validatieaanpak te toetsen. Die verkennende stap diende niet als eindarchitectuur, maar als technische voorbereiding op de plugin-gebaseerde oplossing. Binnen deze bachelorproef geldt een volledig self-hosted en open-source plugin-aanpak binnen CTFd als de centrale oplossingsrichting.

1.4. Scope en afbakening

Dit onderzoek beperkt zich tot on-premise, self-hosted omgevingen. Kubernetes wordt bewust niet als primaire oplossing gekozen omwille van de operationele complexiteit die niet in verhouding staat met de schaal van een doorsnee CTF-event. Cloudplatforms en betaalde diensten vallen buiten scope. De focus ligt op open-source tooling die draait op standaard Linux-servers.

1.5. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt de achtergrond en het begrippenkader geschetst: wat zijn CTF-events, hoe werkt CTFd, wat zijn de relevante Docker-concepten en welke beveiligingsaspecten spelen een rol?

In Hoofdstuk 3 wordt de onderzoeksmethodologie toegelicht.

In Hoofdstuk 4 wordt de huidige situatie (AS-IS) geanalyseerd: hoe worden CTF-events vandaag beheerd en wat zijn de tekortkomingen?

In Hoofdstuk 5 wordt de gewenste situatie (TO-BE) beschreven, inclusief de architectuurvisie en de vereisten voor de oplossing.

In Hoofdstuk 6 wordt de proof-of-concept uitgewerkt. Eerst komt de verkennende standalone PoC kort aan bod als tussenstap, daarna volgt de plugin-PoC als hoofdresultaat van het onderzoek.

In Hoofdstuk 7 worden de testresultaten besproken. Daarbij wordt een onderscheid gemaakt tussen de standalone pre-validatie van ontwerpkeuzes en de uiteindelijke validatie van de plugin-integratie.

In Hoofdstuk 8, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen, samen met aanbevelingen voor toekomstig werk.

2

Achtergrond en begrippenkader

Dit hoofdstuk biedt alle achtergrondkennis die nodig is om de rest van de bachelorproef te begrijpen. Iemand die niet vertrouwd is met CTF-events, Docker of containerbeveiliging, beschikt na het lezen van dit hoofdstuk over een solide basis. Achtereenvolgens komen CTF-events, het CTFd-platform, Docker en containerisatie, containerbeveiliging en load balancing zonder Kubernetes aan bod.

2.1. Capture The Flag (CTF)

Een Capture The Flag (CTF)-event is een cybersecuritywedstrijd waarbij deelnemers technische uitdagingen moeten oplossen om een geheime reeks tekens, de zogenaamde *flag*, te bemachtigen en in te dienen voor punten (Innovirtuoso, [z.d.](#)). CTF-events worden ingezet als educatief middel, als rekruteringsstool en als competitieve sport binnen de cybersecuritygemeenschap.

2.1.1. Types challenges

CTF-challenges zijn doorgaans opgedeeld in categorieën:

- **Web:** kwetsbaarheden in webapplicaties zoals SQL-injectie, Cross-Site Scripting (XSS) en onveilige autorisatie.
- **Pwn (Binary Exploitation):** het exploiteren van kwetsbaarheden in gecompileerde binaire bestanden, bijv. buffer overflows.
- **Reverse Engineering:** analyse en begrip van gecompileerde software zonder broncode.
- **Cryptografie:** het kraken of omzeilen van cryptografische systemen en protocollen.

- **Forensics:** het reconstrueren van gebeurtenissen uit digitale artefacten zoals logbestanden, geheugenafbeeldingen of netwerkopnamen.

2.1.2. Deployment van challenges: statisch vs. dynamisch

Challenges kunnen op twee manieren worden uitgerold. Bij een **statische deployment** wordt één gedeelde instantie van de challenge opgezet die alle deelnemers tegelijk gebruiken. Dit is eenvoudig te beheren, maar brengt risico's mee: één deelnemer kan de omgeving voor alle anderen beïnvloeden. Bij een **dynamische deployment** krijgt elk team of elke gebruiker een eigen, geïsoleerde containerinstantie. Dit verhoogt de integriteit van de wedstrijd maar vereist automatisering (CTFd, [z.d.](#)).

2.1.3. Bekende CTF-platformen

De meest gebruikte platformen voor het organiseren van CTF-events zijn:

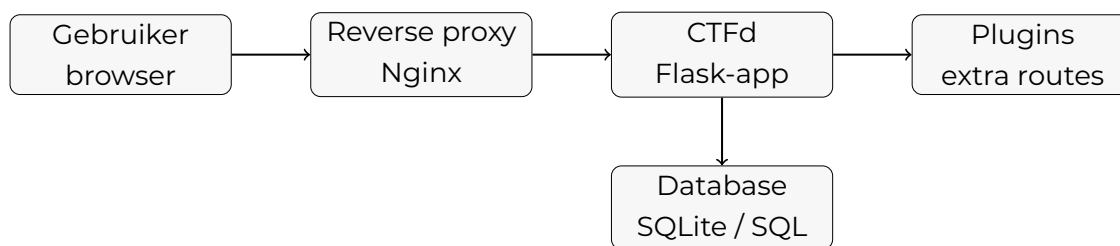
- **CTFd:** het meest wijdverspreide open-source platform, met ondersteuning voor plugins (CTFd LLC, [z.d.-c](#)).
- **rCTF:** een alternatief platform ontwikkeld door het redpwn-team (redpwn, [z.d.](#)).
- **kCTF:** een door Google ontwikkeld Kubernetes-gebaseerd platform, ontworpen voor grote-schaal events (Google, [z.d.](#)).

2.2. CTFd als platform

CTFd is een open-source CTF-platform, gebouwd in Python met het Flask-framework (CTFd LLC, [z.d.-c](#), [z.d.-f](#)). Het biedt een gebruiksvriendelijke interface voor organisatoren en deelnemers. Daarnaast is het modulair uitbreidbaar via een plugin-systeem.

2.2.1. Architectuur van CTFd

CTFd bestaat uit een Flask/WSGI-backend, een SQL-database (standaard SQLite, in productie vaak MySQL of MariaDB) en een webfrontend met JavaScript en CSS (CTFd LLC, [z.d.-b](#), [z.d.-e](#), [z.d.-f](#)). De applicatie kan achter een reverse proxy zoals Nginx draaien (CTFd LLC, [z.d.-a](#)). De API-routes zijn als afzonderlijke componenten uitgewerkt en de REST API ondersteunt de belangrijkste interacties met het platform (CTFd LLC, [z.d.-b](#), [z.d.-d](#)).



Figuur 2.1: Vereenvoudigde architectuur van CTFd. De Flask-app verwerkt de wedstrijdlogica en spreekt zowel de database als optionele plugins aan.

2.2.2. Het plugin-systeem

CTFd ondersteunt plugins die extra functionaliteit toevoegen aan het platform. Een plugin is een Python-pakket dat in de CTFd/plugins/-map geplaatst wordt. Plugins kunnen nieuwe challenge-types registreren, routes toevoegen en bestaande logica uitbreiden (CTFd LLC, [z.d.-c](#), [z.d.-g](#)).

2.2.3. Challenge-beheer en granulariteit

CTFd ondersteunt standaard statische challenges. Voor dynamische instanties, waarbij elke gebruiker of elk team een eigen containerinstantie krijgt, zijn plugins vereist, zoals de CTFd-Whale plugin (Glzjin, [z.d.](#)). De granulariteit (container per gebruiker of per team) is een ontwerpbeslissing met impact op resource-gebruik en isolatie.

2.3. Docker en containerisatie

2.3.1. Wat is Docker?

Docker is een platform voor het bouwen, distribueren en draaien van applicaties in containers (Docker Inc., [z.d.-e](#); Muzumdar e.a., 2024). Een container is een geïsoleerd, lichtgewicht proces dat gebruikmaakt van Linux-kernelmechanismen (namespaces, cgroups) om te worden afgeschermd van andere processen en van de host.

Containers versus virtuele machines: waar een virtuele machine een volledig besturingssysteem emuleert, deelt een container de kernel van de host. Dit maakt containers veel sneller op te starten en minder resource-intensief, maar vergt zorgvuldigere beveiligingsconfiguratie.

2.3.2. Docker Engine API

De Docker Engine API is een RESTful HTTP-API die het mogelijk maakt om programmatisch te communiceren met de Docker-daemon (Docker Inc., [z.d.-b](#)). Via deze API kunnen containers worden aangemaakt, gestart, gestopt en verwijderd, en kunnen netwerken en volumes worden beheerd. De API is de basis voor alle Docker-tooling, inclusief de Docker CLI.

2.3.3. Docker SDK voor Python

De Docker SDK voor Python is een officiële client-bibliotheek die de Docker Engine API omhult en in Python aanroept (Docker Inc., [z.d.-c](#)). Het is de aanbevolen manier om Docker programmatisch te besturen vanuit Python-applicaties, zoals een CTFd-plugin.

Een verwante vraag is of dezelfde aanpak ook met Podman zou werken. Podman biedt in bepaalde configuraties een Docker-compatibele API-socket, waardoor sommige Docker-clients beperkt bruikbaar kunnen zijn. Deze bachelorproef valideert dat echter niet expliciet: de implementatie en de tests richten zich op Docker Engine en de Docker SDK voor Python. Podman-compatibiliteit blijft daarom buiten de bewezen scope van de PoC.

2.3.4. Docker Compose

Docker Compose is een tool voor het definiëren en draaien van multi-container applicaties via een YAML-configuratiebestand (Docker Inc., [z.d.-a](#)). Voor challenge-configuraties biedt Compose een declaratieve manier om containerparameters (image, netwerk, resource-limieten) te definiëren.

2.3.5. Resource management

Docker biedt mechanismen voor het beperken van resource-gebruik per container, gebaseerd op Linux cgroups:

- CPU-limieten: `-cpus` en `-cpu-shares`
- Geheugenlimieten: `-memory` en `-memory-swap`
- I/O-limieten via `blkio-cgroup-parameters`

2.3.6. Netwerkisolatie

Docker biedt verschillende netwerkmodi:

- **Bridge:** de standaardmodus waarbij containers communiceren via een virtuele brug, maar gescheiden zijn van de host.
- **None:** geen netwerktoegang.
- **Custom overlay/bridge networks:** containers kunnen worden ingedeeld in geïsoleerde virtuele netwerken.

2.4. Beveiliging van containers

2.4.1. Beveiligingsrisico's bij dynamisch aanmaken van containers

Wanneer containers dynamisch worden aangemaakt op basis van gebruikersinput, ontstaan extra risico's: een kwaadaardige deelnemer kan proberen de con-

figuratie te manipuleren om bevoorrechte toegang te krijgen of om resources te monopoliseren.

2.4.2. Direct toepasbare basismaatregelen

De OWASP Docker Security Cheat Sheet (OWASP, 2024) en artikelen over containerbeveiliging (Aikido, 2025) identificeren een reeks basismaatregelen:

- **Privilegebeheer:** containers draaien standaard als root; dit wordt bij voorkeur vermeden via `-user`.
- **Read-only bestandssysteem:** via `-read-only` worden schrijfacties beperkt.
- **Capabilities beperken:** Linux capabilities die niet nodig zijn (bijv. `NET_ADMIN`, `SYS_ADMIN`) worden verwijderd via `-cap-drop ALL`.
- **Seccomp-profielen:** beperken de systeemoproepen die een container mag uitvoeren.
- **AppArmor/SELinux:** MAC-profielen die extra toegangscontrole bieden.
- **No-new-privileges:** de optie `-security-opt no-new-privileges` voorkomt privilege-escalatie.

Het CIS Docker Benchmark (Center for Internet Security, 2023) biedt een uitgebreide checklist van aanbevolen beveiligingsinstellingen voor Docker-hosts en -containers.

2.4.3. Container escape

Een container escape is een aanval waarbij een kwaadaardige actor de isolatielaag van de container doorbreekt en toegang verkrijgt tot de host of andere containers. Bekende technieken maken gebruik van kwetsbaarheden in de Docker-daemon, misbruik van gemounte sockets (`/var/run/docker.sock`) of misconfiguraties (bijv. *privileged mode*) (OWASP, 2024).

2.4.4. Resource exhaustion en DoS-risico's

Zonder resource-limieten kan een deelnemer de hostserver overbelasten door een excessief aantal containers te starten of door een container alle CPU- en geheugen-capaciteit te laten consumeren. Dit vormt een *Denial-of-Service* (DoS)-risico voor alle overige deelnemers.

2.5. Load balancing zonder Kubernetes

2.5.1. Waarom geen Kubernetes?

Kubernetes is het de-facto standaardplatform voor containerorkestratie op schaal (Google, [z.d.](#); The Kubernetes Authors, [z.d.](#)). Het biedt ingebouwde load

balancing, automatische schaling en zelfherstel. Voor kleinschalige CTF-events wegen de voordelen echter niet op tegen de operationele complexiteit: Kubernetes vereist een substantieel cluster, diepgaande kennis en aanzienlijke overhead voor beheer.

Een volledige Kubernetes-gebaseerde oplossing zou bovendien een afzonderlijk onderzoeksproject kunnen vormen. Dan moeten onder meer clusterprovisioning, netwerkpolities, ingressconfiguratie, secrets management, node-isolatie, resourcequota en operationele monitoring worden uitgewerkt. Omdat de onderzoeksvraag focust op een lichtgewicht self-hosted aanpak zonder Kubernetes als vereiste, wordt Kubernetes hier enkel als alternatief besproken en niet als implementatierichting uitgewerkt.

2.5.2. Alternatieven

- **Docker Swarm:** Docker's ingebouwde orkestratielaag voor meerdere nodes (Docker Inc., [z.d.-d](#)). Eenvoudiger dan Kubernetes, maar minder uitgebreid.
- **Traefik:** een dynamische reverse proxy die automatisch routes configureert op basis van Docker-labels (Traefik Labs, [z.d.](#)). Ideaal als *entry point* voor dynamisch aangemaakte containers.
- **HAProxy:** een betrouwbare, performante TCP/HTTP load balancer (HAProxy Technologies, [z.d.](#)) voor geavanceerdere routeringsconfiguraties.
- **Nginx:** een veelgebruikte webserver die ook als reverse proxy en load balancer kan fungeren.

2.5.3. Wanneer is schaling nodig?

Bij CTF-events varieert de belasting sterk: er zijn piekbelastingen bij de start van een event wanneer veel deelnemers tegelijk challenges starten. De infrastructuur moet deze piekbelasting kunnen opvangen zonder de stabiliteit van het platform te compromitteren.

3

Methodologie

Dit onderzoek combineert een literatuurstudie met praktijkgericht werk. Het verloopt in vier opeenvolgende fasen.

Fase 1 – Literatuurstudie en begrippenkader (Hoofdstuk 2): Via literatuurstudie worden de relevante concepten en de state-of-the-art in kaart gebracht. De focus ligt op CTFd, de Docker Engine API, containerbeveiliging en load balancing op on-premise infrastructuur.

Fase 2 – Analyse van de huidige en gewenste situatie (Hoofdstukken 4 en 5): De huidige aanpak voor het beheren van CTF-challenges op eigen infrastructuur wordt geanalyseerd (AS-IS). Op basis hiervan en de literatuurstudie worden functionele en niet-functionele vereisten opgesteld en een architectuurvisie uitgetekend (TO-BE).

Fase 3 – Iteratieve proof-of-concept (Hoofdstuk 6): De praktische uitwerking gebeurt in twee stappen. Eerst wordt een beperkte standalone PoC gebouwd om container lifecycle management, opslag van runtime-metadata, cleanup na timeout en basisbeveiligingsmaatregelen los van CTFd te valideren. Op basis van die bevindingen volgt vervolgens de eigenlijke plugin-PoC in CTFd. Die plugin beheert het starten en stoppen van challenge-containers vanuit het platform, biedt configureerbare limieten per challenge en sluit beter aan bij de centrale onderzoeksvraag.

Fase 4 – Validatie en testing (Hoofdstuk 7): De validatie combineert geautomatiseerde tests, smoke tests en gerichte experimenten. De standalone stap levert pre-validatie van ontwerpkeuzes zoals idempotent starten, timeout cleanup, netwerkisolatie en resourcebeperking. De plugin-PoC wordt daarna end-to-end gevalideerd binnen een werkende CTFd-opstelling, zowel in same-host modus als in split-host modus. Load testing met Locust (Locust contributors, [z.d.](#)) wordt gebruikt als indicatie van capaciteitsgedrag, terwijl productierijpe remote Docker-beveiliging, fault injectie en meerhost-schaalverdeling toekomstig werk blijven.

4

Huidige situatie

Dit hoofdstuk analyseert hoe CTF-events vandaag worden georganiseerd op eigen infrastructuur. Het brengt de problemen en beperkingen van de huidige aanpak in kaart als vertrekpunt voor de gewenste oplossing.

4.1. Organisatie van CTF-events op eigen infrastructuur

Bij een on-premise CTF-opzet draait een centraal platform (zoals CTFd) voor gebruikersbeheer, challenges en scoring. Challenges die enkel uit een beschrijving en een download bestaan, vergen weinig infrastructuur. Service-gebaseerde challenges (bijv. een webapplicatie die aangevallen moet worden) vereisen echter een runtime-omgeving die voor deelnemers bereikbaar is.

In veel opzetten wordt die runtime-omgeving gescheiden van het platform beheerd: de organisator bouwt een Docker-image en start één of meerdere containers op een Docker-host. De deelnemer krijgt vervolgens een URL of een host / poortcombinatie om de challenge te bereiken. De documentatie van CTFd beschrijft dit principe expliciet: de challenge-service wordt extern uitgerold en daarna gelinkt vanuit het platform (CTFd, [z.d.](#)). Op basis van gesprekken en beschikbare interne documentatie blijkt dat deze werkwijze vaak handmatig of met beperkte automatisering wordt uitgevoerd (Singier, [2025](#)).

4.2. Manuele container deployments

Wanneer challenge-containers manueel worden uitgerold, gebeurt dit door gaans via `docker run` of `docker compose`. Organisatoren moeten daarbij onder meer Docker-images beheren, poortmapping instellen, DNS- of reverse-proxyconfiguratie voorzien en containers opruimen na afloop. Tijdens een event, met piekbelasting en veel gelijktijdige deelnemers, zorgt dit voor operationele overhead en een verhoogde kans op fouten.

Bovendien ontbreken bij manuele uitrol vaak centrale beleidsregels. Zonder consistente standaardinstellingen voor resource-limieten en netwerkisolatie ontstaat het risico dat één of meerdere containers de host overbelasten of dat deelnemers elkaars instanties kunnen beïnvloeden. Ook het correct en tijdig verwijderen van containers (bijv. na een timeout) vereist bijkomende scripts en monitoring, wat niet in alle scenario's sluitend is.

4.3. Overzicht van bestaande tools en hun tekortkomingen

CTFd is in de basis gericht op het beheren van de wedstrijdlogica en bevat geen ingebouwde orkestratie voor het per gebruiker/team opstarten van service-instanties (CTFd, [z.d.](#); CTFd LLC, [z.d.-c](#)). Voor dynamische deployments bestaan er community-plugins, zoals CTFd-Whale (Glzjin, [z.d.](#)), die proberen containerbeheer te integreren in het platform.

Hoewel dergelijke oplossingen een goed vertrekpunt vormen, voldoet een bestaande tool niet automatisch aan de randvoorwaarden van dit onderzoek: volledig self-hosted inzetbaar, zonder terugkerende kosten, met een voorkeur voor open-source componenten en met voldoende controle over beveiligingsinstellingen en afhankelijkheden. Een alternatief is het gebruik van eigen scripts en conventies, maar dan ontbreekt doorgaans een nauwe koppeling met gebruikersacties in CTFd en is het moeilijker om uniforme beleidsregels (isolatie, quota, timeouts) af te dwingen.

Tot slot bestaan er gehoste of cloud-gebaseerde platformen, maar die vallen buiten de scope van deze bachelorproef wanneer organisaties omwille van privacy, controle of koststructuur op eigen hardware willen blijven.

4.4. Conclusie: wat ontbreekt er?

Uit de analyse volgt dat de huidige aanpak vooral tekortschiet op het vlak van automatisering en afdwingbaarheid. Concreet ontbreekt er een mechanisme dat:

- de containerlevenscyclus rechtstreeks koppelt aan acties in CTFd (start/stop, timeouts en automatisch opruimen);
- beveiligings- en resourcebeleid standaard toepast (CPU/RAM-limieten, netwerkisolatie en veilige standaardinstellingen);
- schaalbaar omgaat met piekbelasting zonder dat organisatoren manueel moeten ingrijpen;
- beheer en monitoring vereenvoudigt (overzicht van actieve instanties en toewijzing aan gebruiker/team).

Deze ontbrekende elementen vormen de input voor de gewenste situatie (TO-BE) en de ontwerpkeuzes van de proof-of-concept.

5

Gewenste situatie

Op basis van de tekortkomingen die in het vorige hoofdstuk werden geïdentificeerd, beschrijft dit hoofdstuk de vereisten voor de oplossing en de architectuurvisie voor het gewenste systeem.

5.1. Vereisten voor de oplossing

5.1.1. Functionele vereisten

De gewenste oplossing automatiseert het beheer van service-gebaseerde challenges. Concreet moet het systeem:

- bij het starten van een challenge automatisch een containerinstantie voorzien per gebruiker of per team;
- de volledige containerlevenscyclus beheren (aanmaken, starten, stopzetten, verwijderen en opruimen na timeout);
- per challenge configureerbare resource-limieten afdwingen (CPU en RAM);
- netwerkisolatie afdwingen tussen instanties van gebruikers of teams;
- ongewenste laterale beweging tussen instanties verhinderen;
- een mechanisme voorzien om het aantal actieve instanties te limiteren (per challenge en/of globaal) om resource-uitputting te vermijden;
- organisatoren een beheerbare manier bieden om challenge-parameters te configureren (Docker-image, poorten, timeouts, limieten);
- een overzicht bieden van actieve instanties, inclusief eigenaar (gebruiker/team), status en vervaltijd, met voldoende logging voor foutopsporing.

5.1.2. Niet-functionele vereisten

Naast de functionele vereisten gelden de volgende niet-functionele vereisten:

- **Self-hosted en open-source:** de oplossing moet inzetbaar zijn op eigen infrastructuur, zonder afhankelijkheid van betaalde diensten.
- **Beveiliging:** veilige standaardinstellingen en een minimale aanvalsoppervlakte, met focus op isolatie en het vermijden van container-escapes (Center for Internet Security, [2023](#); OWASP, [2024](#)).
- **Geen Kubernetes als vereiste:** de basiswerking mag niet afhankelijk zijn van Kubernetes, om de operationele complexiteit te beperken.
- **Schaalbaarheid:** de opzet moet piekbelasting kunnen opvangen en uitbreidbaar zijn naar meerdere hosts of VM's.
- **Beheerbaarheid en reproduceerbaarheid:** installatie- en beheerinstructies, consistente configuratie en voldoende logging/monitoring.
- **Modulariteit:** uitbreidbaar naar extra challenge-types of extra beleidsregels zonder ingrijpende aanpassingen aan de kernlogica.

5.2. Architectuurvisie

De oplossing wordt uitgewerkt als een uitbreiding op CTFd via het plugin-systeem (CTFd LLC, [z.d.-c](#), [z.d.-g](#)). De plugin fungeert als orkestratielaag die gebruikersacties in het platform vertaalt naar containeracties op een Docker-host. Daarmee blijft de gebruikerservaring in CTFd gecentraliseerd, terwijl de challenge-runtime via Docker wordt beheerd.

5.2.1. Componenten en verantwoordelijkheden

- **CTFd:** voorziet authenticatie, challenge-definities en de gebruikersinteractie.
- **CTFd-plugin:** beheert de containerlevenscyclus, houdt de toewijzing bij (gebruiker/team → container) en past beleidsregels toe (timeouts, quota, resource-limieten en isolatie).
- **Docker-host:** draait de challenge-containers en wordt aangestuurd via de Docker Engine API (Docker Inc., [z.d.-b](#)). In de PoC gebeurt dit via de Docker SDK voor Python (Docker Inc., [z.d.-c](#)).
- **Toegangspunt voor deelnemers:** in de huidige PoC gebeurt toegang via gepubliceerde hostpoorten. Een reverse proxy blijft een relevante uitbreidingsrichting, maar de architectuur moet ook zonder die tussenlaag bruikbare runtime-URL's kunnen genereren voor same-host en split-host deployments.

5.2.2. Datastromen op hoofdlijnen

- **Start:** een deelnemer start een challenge → de plugin valideert beleidsregels (quota en timeout) → maakt een container aan met limieten en isolatie → publiceert een toegangspunt → de deelnemer krijgt een URL of verbindingsinformatie.
- **Stop en opruimen:** bij expliciet stopzetten of bij het verlopen van de timeout worden container en gerelateerde resources (netwerk, metadata) verwijderd om resourcelekken te vermijden.
- **Afsluiten bij solve:** wanneer een challenge correct opgelost wordt, stopt de plugin de actieve runtime van die gebruiker of dat team automatisch zodat de infrastructuur niet onnodig blijft draaien.

5.3. Granulariteitsbeslissing: container per gebruiker of per team

De keuze tussen een instantie per gebruiker of per team beïnvloedt zowel isolatie als resourceverbruik:

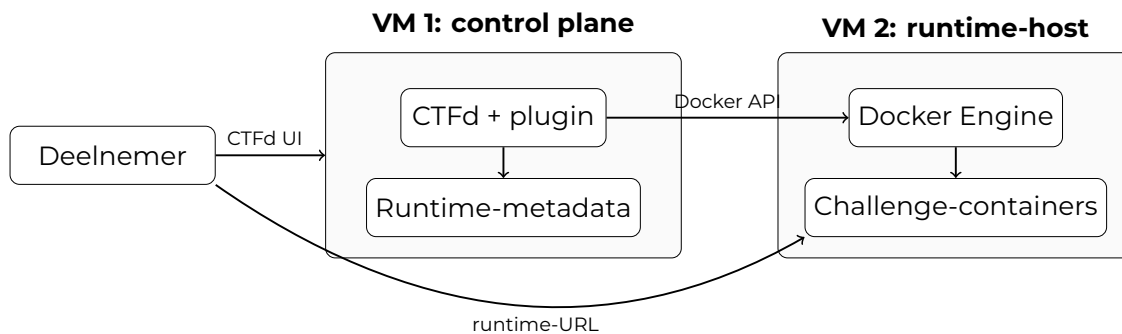
Tabel 5.1: Vergelijking tussen een containerinstantie per gebruiker en per team.

Criteria	Per gebruiker	Per team
Isolatie	Maximale scheiding tussen deelnemers; weinig risico op onderlinge interferentie.	Lagere scheiding omdat teamleden dezelfde omgeving delen.
Resourceverbruik	Meer containers en hogere piekbelasting bij grotere events.	Minder instanties en dus lagere infrastructuurdruk.
Operationele eenvoud	Meer lifecycle-events om te beheren, maar eenduidige toewijzing per deelnemer.	Minder instanties om te beheren, maar extra logica nodig om teamtoegang correct af te handelen.
Geschikte context	Individuele events of challenges waarbij elke deelnemer een eigen sandbox nodig heeft.	Teamgebaseerde events waar samenwerking binnen dezelfde runtime gewenst is.

Binnen de PoC wordt uitgegaan van een flexibele aanpak: wanneer het event in teammodus verloopt, wordt een instantie per team voorzien; bij individuele deelname wordt een instantie per gebruiker voorzien. Hierdoor blijft de isolatie passend bij het wedstrijdmodel, terwijl onnodige containerexplosie wordt vermeden.

5.4. Schaalbaarheidsvereisten

De infrastructuur moet bestand zijn tegen piekbelasting, bijvoorbeeld wanneer veel deelnemers gelijktijdig een service-challenge starten. Figuur 5.1 toont de gewenste deploymentvariant waarin het control plane en de challenge-runtime logisch gescheiden zijn.



Figuur 5.1: Gewenste split-host architectuur. CTFd en de plugin blijven het control plane, terwijl challenge-containers op een aparte runtime-host draaien.

Dit vertaalt zich naar de volgende vereisten:

- **Piekbestendig opstartpad:** het systeem moet een piek aan containercreatie kunnen verwerken zonder dat het platform onbeschikbaar wordt.
- **Begrenzing en evenwichtige verdeling:** limieten op actieve instanties en afdwingbare quota vermijden dat één challenge of deelnemer disproportioneel resources consumeert.
- **Deployment-variant met twee VM's:** de oplossing moet kunnen werken met een control plane op een eerste VM (CTFd + plugin) en een afzonderlijke runtime-host op een tweede VM (Docker + challenge-containers). Zo blijven wedstrijdplatform en challenge-executie logisch gescheiden.
- **Gescheiden control- en datapad:** de plugin moet enerzijds een Docker-control endpoint kunnen aanspreken en anderzijds correcte spelergerichte runtime-URL's kunnen genereren. In een split deployment zijn die twee adressen niet noodzakelijk gelijk.
- **Begrensde publicatie van poorten:** wanneer challenge-services via host-poorten beschikbaar worden gemaakt, moet een configureerbaar poortbereik mogelijk zijn zodat firewallregels, audit en troubleshooting beheersbaar blijven.
- **Beveiligde remote aansturing:** communicatie met een remote Docker-host moet in een productiecontext beschermd worden, bij voorkeur via TLS en strikte netwerksegmentatie.

- **Horizontale uitbreiding:** de architectuur moet uitbreidbaar zijn naar meerdere Docker-hosts. Voor taakverdeling over meerdere hosts kan Docker Swarm een mogelijke optie zijn (Docker Inc., [z.d.-d](#)), maar dit valt buiten de huidige PoC.
- **Dynamische routing:** voor een latere productie-opzet kan een reverse proxy nuttig zijn om dynamisch naar nieuw aangemaakte instanties te routeren (Traefik Labs, [z.d.](#)). Ook dit behoort tot toekomstig werk.

Concreet betekent dit dat de gewenste architectuur niet mag vertrekken van een hard gekoppelde lokale Docker-socket. Ze moet expliciet rekening houden met twee deploymentmodi: een eenvoudige same-host opstelling voor kleine events en een split-host opstelling voor betere scheiding tussen platform en runtime. Verdere onderzoeksvragen verschuiven daarmee van “kan dit technisch werken?” naar “hoe wordt deze koppeling productierijp gemaakt op vlak van beveiliging, fouttolerantie en verdeling over meerdere runtime-hosts?”.

6

Proof-of-Concept

Dit hoofdstuk beschrijft de praktische uitwerking van het onderzoek. De nadruk ligt op de uiteindelijke CTFd-plugin, omdat die rechtstreeks aansluit bij de onderzoeksvraag: challenge-containers dynamisch beheren vanuit CTFd. Voorafgaand daaraan werd echter een beperkte standalone PoC gebouwd om de kernmechanismen sneller te testen. Die verkennende stap wordt kort besproken omdat ze verschillende ontwerpkeuzes voor de plugin heeft onderbouwd.

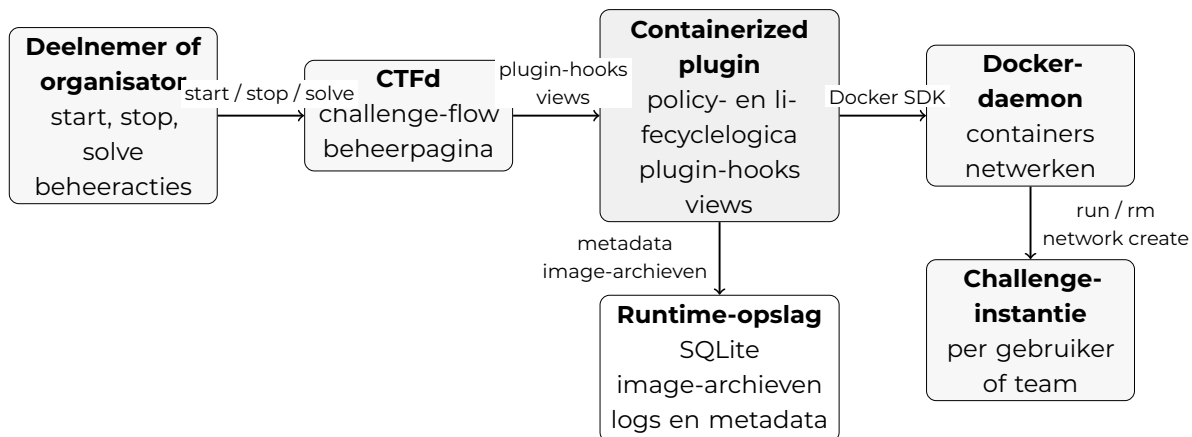
6.1. Architectuuroverzicht

De uiteindelijke PoC bestaat uit een CTFd-instantie waarin een eigen plugin een nieuw challenge-type toevoegt. Die plugin spreekt de Docker-daemon aan via de Docker SDK voor Python (Docker Inc., [z.d.-c](#)) en beheert de volledige levenscyclus van per-gebruiker of per-team challenge-instanties. Dezelfde plugin kan zowel een lokale Docker-daemon via de Unix-socket aanspreken als een remote Docker API op een andere host. Runtime-metadata, zoals actieve instanties en image-archieven, worden lokaal bijgehouden zodat de plugin ook cleanup en administratieve acties kan uitvoeren.

De opzet bevat vier kernonderdelen:

- **CTFd**: het wedstrijdplatform met authenticatie, challenges en scoreverwerking.
- **CTFd-plugin**: de eigenlijke orkestratielaag die start-, stop- en cleanupacties uitvoert.
- **Docker-daemon**: de runtime waarin challenge-containers en hun netwerken worden aangemaakt; in split-host modus draait die op een afzonderlijke host.
- **Runtime-opslag**: een lichte SQLite-opslag voor actieve instanties, logs en metadata van geuploade image-archieven.

Tijdens de implementatie werd eerst vertrokken van een single-host opstelling om de integratie beheersbaar te houden. Nadien is dezelfde architectuur uitgebreid naar een split-host model waarin het Docker-control endpoint en de spelergerichte runtime-host afzonderlijk configureerbaar zijn. Binnen de validatie van deze bachelorproef werd die tweede modus zowel lokaal gesimuleerd als op twee afzonderlijke VM's getest.



Figuur 6.1: Architectuur van de plugin-PoC. CTFd blijft het centrale platform, terwijl de plugin de lifecycle van challenge-containers en de bijhorende metadata beheert. De Docker-daemon kan lokaal of op een afzonderlijke runtime-host draaien.

6.2. Verkennende standalone PoC

Voor de plugin werd uitgewerkt, werd eerst een kleine standalone orchestrator gebouwd in Flask. Het doel daarvan was niet om een alternatieve eindarchitectuur naar voren te schuiven, maar om drie vragen snel te beantwoorden:

- Kan een Python-service containers starten, stoppen en opruimen via de Docker SDK?
- Welke basisbeveiligingsmaatregelen zijn praktisch toepasbaar zonder de challenge-flow onnodig complex te maken?
- Hoe kunnen lifecycle-regels zoals timeouts, quota en idempotent starten eenvoudig gevalideerd worden?

De standalone PoC bood een challenge-registry, een eenvoudige webinterface, API-endpoints voor start en stop, een timeout-reaper en een beperkte logginglaag. Uit die stap volgden een aantal keuzes die rechtstreeks in de plugin zijn meegenomen:

- gebruik van de Docker SDK in plaats van shell-calls naar de Docker CLI;
- idempotent startgedrag voor dezelfde gebruiker of hetzelfde team;

- standaardresourceprofielen per challenge;
- cleanup van containers en netwerken bij timeout of expliciete stop;
- netwerkisolatie per instantie.

De standalone PoC was dus vooral een technische voorstudie die toeliet om de risicovolste runtime-aspecten vroeg te toetsen. Zodra die basis werkte, werd de focus verschoven naar de CTFd-plugin, omdat die nauwer aansluit bij het doel van dit onderzoek: de containerlevenscyclus rechtstreeks koppelen aan acties in het CTF-platform.

Tabel 6.1: Vergelijking tussen de verkennende standalone PoC en de uiteindelijke CTFd-plugin PoC.

Aspect	Standalone PoC	CTFd-plugin PoC
Hoofddoel	Snelle validatie van lifecycle-mechanismen en beveiligingsmaatregelen.	Integratie van die mechanismen in de echte challenge-flow van CTFd.
Interface	Eigen Flask-dashboard en REST-API.	Challenge-type en beheerpagina binnen CTFd.
Bewijswaarde	Toont dat Docker-orkestratie technisch werkt in isolatie.	Toont dat gebruikersacties in CTFd rechtstreeks runtimegedrag kunnen sturen.
Belangrijkste output	Inzichten over idempotent starten, timeout cleanup en isolatie.	Werkende plugin met uploadflow, solve-cleanup en administratieve controle.
Rol in deze bachelorproef	Verkennende tussenstap om ontwerpkeuzes te onderbouwen.	Hoofddresultaat dat de onderzoeksvraag het dichtst benadert.

6.3. Implementatie van de CTFd-plugin

6.3.1. Custom challenge-type en configuratie

De plugin voegt een nieuw challenge-type containerized toe aan CTFd. Een organisator kan zo per challenge runtime-parameters configureren zonder externe scripts of manuele docker run-commando's. De belangrijkste parameters zijn:

- Docker-image of geupload image-archief;

- een of meerdere containerpoorten;
- CPU-limiet;
- geheugenlimiet;
- timeout;
- maximum aantal gelijktijdige instanties.

Naast een klassieke image-referentie ondersteunt de plugin ook het uploaden van een `.tar`, `.tar.gz` of `.tgz`-archief uit `docker save`. Dat archief wordt lokaal opgeslagen, in de Docker-daemon geïmporteerd en aan de challenge gekoppeld. Daardoor kan een organisator ook zonder externe registry een image beschikbaar maken voor de PoC. Wanneer de lokaal geïmporteerde image later ontbreekt, kan de plugin het gekoppelde archief automatisch opnieuw importeren bij een volgende runtime-start. Die eigenschap bleek bijzonder nuttig in de smoke tests, omdat het daarmee mogelijk werd om de volledige upload- en herimportflow reproduceerbaar te valideren.

6.3.2. Start- en stopflow voor deelnemers

Wanneer een deelnemer een containerized challenge opent, kan die via het challenge-scherm een eigen runtime-instantie starten. De plugin vertaalt die actie naar een Docker-start met de juiste challenge-configuratie. In individuele modus gebeurt dit per gebruiker; in teammodus per team. Daarbij gelden twee belangrijke beleidsregels:

- voor dezelfde gebruiker of hetzelfde team wordt een bestaande actieve instantie hergebruikt in plaats van een nieuwe te starten;
- zodra het ingestelde maximum aantal instanties is bereikt, weigert de plugin bijkomende starts.

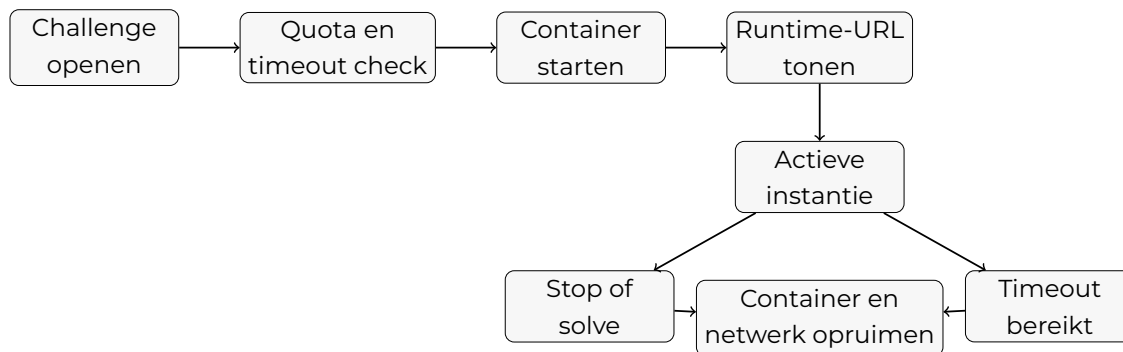
Na een succesvolle start krijgt de deelnemer een of meerdere toegangspunten terug met de gepubliceerde hostpoort(en). Die URL's worden opgebouwd op basis van een publiek configureerbare runtime-host, zodat de plugin ook correct blijft werken wanneer CTFd en Docker niet op dezelfde machine draaien. De plugin bewaart tegelijk metadata zoals eigenaar, starttijd, vervaltijd, container-ID, netwerknaam en poortbindingen. Wanneer de deelnemer expliciet stopt, wordt de container verwijderd en wordt ook het bijhorende Docker-netwerk opgeruimd.

6.3.3. Timeout cleanup en cleanup bij solve

De plugin bevat een achtergrondproces dat periodiek controleert welke instanties vervallen zijn. Wanneer een timeout bereikt is, wordt de container geforceerd

gestopt en worden de runtime-resources vrijgemaakt. De eindstatus van zo'n instantie wordt als verlopen geregistreerd zodat achteraf zichtbaar blijft waarom ze stopte.

Daarnaast grijpt de plugin ook in op het moment dat een challenge correct opgelost wordt. In dat geval wordt de actieve instantie van de betrokken gebruiker of het betrokken team automatisch stopgezet. Die koppeling is belangrijk voor CTF-contexten: een deelnemer hoeft een oplossing niet eerst manueel te stoppen en de infrastructuur blijft niet onnodig resources verbruiken nadat de flag al werd ingediend.



Figuur 6.2: Start-, stop- en cleanupflow voor een containerized challenge. Dezelfde cleanupstap wordt gebruikt bij manueel stoppen, een correcte solve en timeout.

6.3.4. Administratieve runtime-interface

Voor organisatoren voorziet de plugin een aparte beheerpagina binnen CTFd. Daarop zijn twee soorten informatie zichtbaar:

- een overzicht van alle containerized challenges met hun resourceprofiel;
- een runtime-overzicht van actieve en historische instanties.

Vanuit die interface kunnen beheerders een individuele instantie geforceerd stoppen, alle actieve instanties van een challenge stoppen en de timeout-reaper manueel uitvoeren. Dit verhoogt de beheerbaarheid van de PoC aanzienlijk, omdat de organisator niet rechtstreeks op de Docker-host hoeft in te loggen om runtime-problemen op te lossen.

6.4. Beveiliging in de plugin-PoC

De plugin past bij elke containerstart een aantal direct toepasbare basismaatregelen toe, in lijn met de OWASP Docker Security Cheat Sheet (OWASP, 2024) en het CIS Docker Benchmark (Center for Internet Security, 2023). Concreet gaat het om de volgende Docker-instellingen:

- **Read-only root filesystem:** het hoofdbestandssysteem van de container wordt alleen-lezen gemount. De challenge kan daardoor niet zomaar systeembestanden, configuratie of applicatiebestanden in de container aanpassen. Dat beperkt de impact van een exploit die probeert persistentie of extra tooling in de container weg te schrijven.
- **cap_drop=['ALL']:** Linux-capabilities zijn fijnmazige rechten die processen extra kernelmogelijkheden geven, zoals netwerkconfiguratie aanpassen of systeembeheeracties uitvoeren. Door alle capabilities standaard weg te nemen, start de container met zo weinig mogelijk privileges. Alleen wanneer een challenge aantoonbaar extra rechten nodig heeft, zou een organisator daar bewust van kunnen afwijken.
- **security_opt=['no-new-privileges:true']:** deze instelling verhindert dat processen binnen de container via mechanismen zoals setuid-binaries extra privileges verkrijgen. Zelfs wanneer een programma binnen de container normaal gezien met verhoogde rechten zou starten, mag het proces geen nieuwe privileges krijgen bovenop wat het al had.
- **PID-limiet:** het aantal processen dat binnen de container mag draaien wordt begrensd. Dit voorkomt eenvoudige fork bombs of challenges die per ongeluk zeer veel processen aanmaken en zo de Docker-host belasten.
- **CPU- en geheugenlimieten:** per challenge wordt vastgelegd hoeveel CPU-tijd en RAM een container maximaal mag gebruiken. Daardoor kan één foutieve of kwaadwillige challenge-instantie niet alle resources van de host opeisen en andere deelnemers of CTFd zelf verstoren.
- **Tijdelijke tmpfs-mount voor /tmp:** de map /tmp blijft beschrijfbaar, maar wordt in geheugen geplaatst en verdwijnt wanneer de container stopt. Zo kunnen applicaties die tijdelijke bestanden nodig hebben blijven werken, terwijl permanente writes naar het containerbestandssysteem vermeden worden.
- **Bridge-netwerk per runtime-instantie:** elke challenge-instantie krijgt een eigen Docker-netwerk. Containers van verschillende gebruikers of teams delen daardoor geen standaard netwerksegment en kunnen elkaar niet zomaar via containernaam of intern IP-adres ontdekken.

Die combinatie beperkt het risico op privilege-escalatie, laterale beweging en resource-uitputting. De keuze voor per-instantie netwerken zorgt ervoor dat deelnemers niet zomaar andere challenge-instanties kunnen ontdekken of aanspreken. Voor split-host deployments ondersteunt de plugin daarnaast een expliciet configureerbaar gepubliceerd poortbereik, zodat firewallregels op de runtime-host begrensd kunnen blijven. Belangrijk is wel dat deze PoC een basisbeveiliging demonstreert en geen volledig production-ready hardeningprofiel oplevert. Zo is ondersteuning voor aangepaste seccomp-profielen nog geen afgewerkt onderdeel van de implementatie en werd remote Docker-communicatie in de validatie enkel op wegwerpomgevingen zonder TLS getest.

6.5. Deployment- en schaalbaarheidsoverwegingen

De huidige plugin-PoC ondersteunt twee deploymentmodi. In same-host modus draait CTFd samen met de plugin op dezelfde host als de Docker-daemon en wordt de lokale Unix-socket gebruikt. In split-host modus blijft CTFd het control plane, maar wordt de Docker-daemon op een andere host aangesproken via een afzonderlijk control endpoint. De spelergerichte runtime-host kan daarbij verschillen van het Docker API endpoint. Binnen die scope toont de PoC aan dat een plugin-gebaseerde aanpak haalbaar is voor:

- dynamisch starten en stoppen van challenge-containers;
- afdwingen van limieten per challenge;
- automatisch cleanupgedrag bij timeout en solve;
- administratieve controle vanuit CTFd;
- scheiding tussen control plane en runtime-host in een split deployment.

De split-host ondersteuning werd in deze bachelorproef ook effectief gevalideerd. Eerst gebeurde dat lokaal via een Compose-opstelling met een aparte `docker:dind`-service en een `smoke-runner` binnen hetzelfde netwerk. Daarna volgde een reële validatie op twee aparte Lima-VM's: één VM voor CTFd en de plugin, en één VM voor de challenge-containers. Daarbij werden de gepubliceerde challenge-poorten begrensd tot een vast bereik, zodat de runtime-host eenvoudiger afgeschermd kon worden.

Voor verdere schaalvergroting blijven wel nog een aantal vragen over:

- hoe remote Docker-communicatie in productie TLS-beveiligd en operationeel beheersbaar wordt ingericht;
- hoe het systeem reageert wanneer de runtime-host tijdelijk onbereikbaar wordt tijdens start, stop of cleanup;
- hoe dynamische routing best wordt gecombineerd met een reverse proxy in plaats van louter hostpoortmapping;
- hoe meerdere runtime-hosts evenwichtig kunnen worden aangestuurd vanuit dezelfde plugin.

7

Validatie en testing

Dit hoofdstuk bespreekt hoe de oplossing werd gevalideerd. De validatie verloopt in twee lagen. Eerst werd een aantal ontwerpkeuzes vooraf getoetst in de standalone PoC. Daarna werd de uiteindelijke plugin-PoC end-to-end gevalideerd in een werkende CTFd-opstelling. Op die manier wordt een onderscheid gemaakt tussen pre-validatie van technische mechanismen en validatie van de finale integratie.

7.1. Testplan

7.1.1. Doelstellingen

Het testplan had vijf concrete doelen:

- aantonen dat challenge-containers correct gestart, hergebruikt en gestopt worden;
- verifiëren dat timeout cleanup en solve-triggered cleanup werken;
- nagaan of de gekozen basisbeveiligingsmaatregelen effectief op containers worden toegepast;
- observeren hoe de oplossing zich gedraagt onder gelijktijdige startverzoeken;
- bevestigen dat de plugin ook kan werken wanneer CTFd en de Docker-runtime op aparte hosts draaien.

7.1.2. Afbakening

Niet alle oorspronkelijk overwogen validaties zijn in deze fase volledig uitgevoerd. Vooral reverse-proxy-gebaseerde routing, TLS-beveiligde remote Docker-communicatie, uitgebreide foutinjectie en schaalverdeling over meerdere runtime-hosts vallen buiten de effectief afgeronde validatie. Deze punten worden daarom niet als bewezen resultaat voorgesteld, maar als toekomstig werk.

7.2. Pre-validatie via de standalone PoC

De standalone PoC werd gebruikt om fundamentele runtime-mechanismen los van CTFd te testen. De resultaten zijn relevant, omdat dezelfde ontwerpprincipes later in de plugin-PoC zijn toegepast.

7.2.1. Geautomatiseerde tests en API-validatie

Voor de standalone orchestrator zijn geautomatiseerde tests uitgevoerd met `pytest`. Deze tests valideren onder meer:

- registreren van challenge-configuraties;
- idempotent startgedrag per `challenge_id` + `user_id`;
- afdwingen van `max_instances`;
- timeout expiry in de servicelaag;
- het basis happy-path van de API.

De gerapporteerde uitkomst van deze testset was `4 passed in 0.60s`. Hoewel dit nog geen bewijs is voor de CTFd-integratie zelf, wijst het er wel op dat de gekozen lifecycle-logica technisch correct werkt in een afgebakende testomgeving.

7.2.2. End-to-end smoke test

Na de geautomatiseerde tests was nog een beperkte end-to-end controle nodig, omdat de lifecycle-logica pas volledig betekenis krijgt wanneer ook Docker zelf, het netwerk en de gepubliceerde challenge-endpoints betrokken zijn. Daarvoor werd een smoke test uitgevoerd op de standalone PoC. In deze context betekent een smoke test geen volledige bewijsvoering van alle randgevallen, maar een snelle controle of de belangrijkste runtime-flow in een echte Docker-opstelling niet fundamenteel stuk is.

De test bouwde eerst de demo-image en de orchestrator op, registreerde daarna een challenge en startte vervolgens twee challenge-instanties. Voor beide instanties werd gecontroleerd of het gepubliceerde endpoint bereikbaar was. Nadien werden de instanties opnieuw stopgezet. Op die manier controleerde de smoke test in één samenhangende flow of de orchestrator een challenge kan voorbereiden, starten, bereikbaar maken en opruimen. De test bevestigde daarmee dat de kernstroom `start` → `bereikbaar` → `stop` correct werkt buiten een zuiver gemockte of geïsoleerde testomgeving.

7.2.3. Timeout, isolatie en resourcebeperking

Drie gerichte experimenten waren bijzonder bruikbaar als pre-validatie van de pluginkeuzes:

- **Timeout cleanup:** een challenge met een timeout van 30 seconden werd gestart en na 35 seconden opnieuw gecontroleerd. De instantie kreeg correct de status `expired` met stopreden `timeout`.
- **Netwerkisolatie:** twee containers op aparte bridge-netwerken konden elkaar niet via naamresolutie ontdekken, wat de gekozen netwerkaanpak ondersteunt.
- **Hardening flags:** inspectie van een draaiende container toonde aan dat `ReadOnlyRootfs`, `CapDrop`, `SecurityOpt`, geheugenlimiet, CPU-limiet en `PidsLimit` effectief waren toegepast.

7.2.4. Load snapshot met Locust

Locust (Locust contributors, [z.d.](#)) is een open-source loadtestingtool waarbij gebruikersgedrag in Python wordt beschreven. In deze bachelorproef werd het gebruikt om meerdere gelijktijdige gebruikers na te bootsen die API-acties uitvoeren tegen de standalone orchestrator. De gebruikte `locustfile` bevatte twee acties: het starten van een instantie en het opvragen van de actieve instanties. Locust werd in *headless* modus uitgevoerd, dus zonder webinterface, met een vast aantal virtuele gebruikers, een instroomsnelheid en een korte looptijd. Daardoor was de test reproduceerbaar en geschikt als snelle momentopname.

De term *load snapshot* wordt bewust gebruikt in plaats van volledige performance-test. Het doel was niet om maximale schaalbaarheid of productierijpe throughput te bewijzen, maar om te observeren hoe de PoC reageert wanneer meerdere startverzoeken kort na elkaar binnenkomen. In het create-only scenario werden 128 requests uitgevoerd, waarvan 84 faalden met `409 CONFLICT` zodra de ingestelde capaciteit bereikt werd. Die conflicten worden hier niet als technische fout geïnterpreteerd, maar als verwacht gedrag: de API weigert extra instanties wanneer `max_instances` bereikt is. Dit resultaat wijst dus niet op een al grootschalig geoptimaliseerde oplossing, maar laat wel zien dat de capaciteitslimiet effectief wordt afgedwongen in plaats van stilzwijgend overschreden te worden.

7.3. Validatie van de CTFd-plugin

7.3.1. Functionele end-to-end validatie

De plugin-PoC werd eerst gevalideerd met een volledige smoke test in een draaiende CTFd-omgeving in *same-host* modus. Die test doorloopt de belangrijkste gebruikers- en beheerstappen:

- initialiseren van CTFd;
- uploaden van een Docker-archief via de plugin;
- aanmaken van een containerized challenge met runtime-parameters;

- verwijderen van de lokaal geïmporteerde image om herimport uit het geüploade archief te forceren;
- registreren van drie spelers;
- starten van meerdere runtime-instanties;
- controleren of de challenge-service effectief bereikbaar is voor de speler;
- opnieuw starten van dezelfde challenge om te bevestigen dat de bestaande instantie hergebruikt wordt;
- verifiëren dat `max_instances` wordt afgedwongen;
- bevestigen dat een vrijgekomen slot opnieuw gebruikt kan worden na cleanup;
- manueel stopzetten van een runtime;
- indienen van de correcte flag;
- controleren of de runtime na solve automatisch werd opgeruimd;
- controleren of timeout cleanup werkt voor een aparte testchallenge;
- verifiëren via het admin-overzicht dat de stopreden als `challenge-solved` geregistreerd werd.

Deze smoke test is relevant omdat ze laat zien dat de plugin niet enkel losse Docker-acties kan uitvoeren, maar ook correct samenwerkt met de challenge-flow van CTFd.

7.3.2. Split-host validatie

Dezelfde eind-tot-eindflow werd daarna uitgevoerd in een split-host opstelling. Eerst gebeurde dat in een lokale simulatie met twee logische hosts binnen Docker Compose: CTFd draaide samen met de plugin in één service, terwijl een afzonderlijke `docker:dind`-service de remote Docker-host voorstelde. Omdat de gepubliceerde challenge-services in die simulatie enkel zinvol bereikbaar waren vanuit het Compose-netwerk, werd een afzonderlijke smoke-runner container gebruikt om de spelerflow uit te voeren.

Deze validatie toonde aan dat de plugin twee adressen correct van elkaar kan scheiden:

- het Docker-control endpoint dat door de plugin gebruikt wordt om containers te starten en te stoppen;
- de publieke runtime-host die in de access-URL's voor spelers wordt opgenomen.

Daarnaast werd bevestigd dat een configureerbaar gepubliceerd poortbereik nuttig en werkbaar is in split-host modus. De plugin kiest in dat geval een vrije poort binnen het opgegeven bereik in plaats van willekeurig een hostpoort te gebruiken.

7.3.3. Reële validatie op twee VM's

Op 24 maart 2026 werd de split-host architectuur ook gevalideerd op twee afzonderlijke Lima-VM's. In die opstelling draaiden CTFd en de plugin op een eerste VM, terwijl een tweede VM uitsluitend de challenge-containers uitvoerde. Beide VM's waren verbonden via een gedeeld user-v2-netwerk. De smoke test werd vanuit de CTFd-VM uitgevoerd, omdat de macOS-host in deze Lima-configuratie het interne subnet niet rechtstreeks kon bereiken.

De reële VM-validatie bevestigde de belangrijkste technische aannames:

- de plugin kan challenge-containers over een remote Docker API op een andere VM starten, stoppen en opruimen;
- de gegenereerde access-URL's wezen correct naar de runtime-VM;
- de gepubliceerde hostpoorten bleven binnen het ingestelde bereik 20000–20099;
- na afloop bleven geen plugin-beheerde challenge-containers of netwerken achter op de runtime-VM.

7.3.4. Wat de pluginvalidatie laat zien

Op basis van de plugin smoke test kunnen binnen de scope van deze bachelorproef de volgende uitspraken verantwoord worden:

- de plugin kan een eigen challenge-type toevoegen aan CTFd;
- een organisator kan een image-archief uploaden, aan een challenge koppelen en later opnieuw laten importeren;
- een deelnemer kan vanuit CTFd een eigen runtime-instantie starten;
- een tweede startverzoek voor dezelfde gebruiker hergebruikt de bestaande instantie;
- capaciteitslimieten en slothergebruik worden correct afgedwongen;
- een correcte solve triggert automatische cleanup van de actieve runtime;
- timeout cleanup werkt ook binnen de pluginflow;
- de admin-interface houdt de runtimegeschiedenis bij en toont de finale status;
- dezelfde plugincode werkt zowel tegen een lokale Docker-daemon als tegen een remote Docker-host op een andere VM.

7.3.5. Wat bewust niet als bewezen claim wordt opgenomen

De plugin-PoC toont in deze fase nog niet aan dat de oplossing al klaar is voor productie-inzet. In het bijzonder werden de volgende punten niet volledig gevalideerd en worden ze daarom niet als gerealiseerd resultaat voorgesteld:

- TLS-beveiligde remote Docker-communicatie voor persistente of gedeelde omgevingen;
- reverse-proxy-routing naar dynamisch aangemaakte instanties;
- ondersteuning voor aangepaste seccomp-profielen;
- uitgebreide foutinjectie, bijvoorbeeld een tijdelijk onbereikbare Docker-host;
- taakverdeling over meerdere runtime-hosts of een volwaardige meerhost-architectuur.

7.4. Interpretatie van de resultaten

De combinatie van standalone pre-validatie en plugin-validatie biedt voldoende basis om vier observaties te formuleren. Ten eerste is de kern van het probleem technisch oplosbaar: challenge-containers kunnen dynamisch gestart, hergebruikt en opgeruimd worden op basis van acties in CTFd. Ten tweede kunnen direct toepasbare beveiligingsmaatregelen zoals read-only filesystems, capability dropping, no-new-privileges en resource-limieten effectief in de runtime worden opgenomen. Ten derde laat de plugin smoke test zien dat deze runtime-logica bruikbaar kan worden ingepast in het CTFd-platform zelf. Ten vierde toont de split-host validatie aan dat dezelfde pluginarchitectuur ook werkt wanneer CTFd en de Docker-runtime op aparte hosts of VM's draaien.

De validatie maakt tegelijk duidelijk waar de grenzen van de huidige PoC liggen. Productierijpe remote Docker-communicatie, meer geavanceerde hardening, reverse-proxy-integratie en schaalbaarheid over meerdere runtime-hosts zijn nog niet volledig onderzocht. De belangrijkste volgende stap is daarom niet langer het aantonen van split deployment op zich, maar het hardenen en uitbreiden van die opstelling voor realistische productieomgevingen.

8

Conclusie

Deze bachelorproef onderzocht hoe Docker-containers die gelinkt zijn aan CTFd-challenges dynamisch beheerd, aangemaakt en beëindigd kunnen worden op basis van gebruikersacties en ingestelde parameters op on-premise infrastructuur. Op basis van de literatuurstudie, de verkennende standalone PoC en de uiteindelijke plugin-PoC kan geconcludeerd worden dat een plugin-gebaseerde aanpak binnen CTFd hiervoor een haalbare oplossingsrichting is.

8.1. Beantwoording van de onderzoeksvragen

De centrale onderzoeksvraag kan positief beantwoord worden. Het is mogelijk om vanuit CTFd challenge-containers dynamisch te starten en opnieuw te stoppen, zolang de containerlevenscyclus expliciet wordt gekoppeld aan challenge-acties en runtime-beleid. In de uitgewerkte PoC gebeurt dat via een eigen containerized challenge-type dat Docker-containers start met vooraf ingestelde limieten en ze achteraf ook weer opruimt.

Ook de deelvragen kunnen op basis van de resultaten beantwoord worden:

- **Welke beveiligingsmaatregelen zijn minimaal vereist?** Een bruikbaar basisprofiel bestaat uit een read-only root filesystem, het droppen van Linux capabilities, no-new-privileges, CPU- en geheugenlimieten, een PID-limiet en netwerkisolatie per instantie.
- **Hoe worden resource-limieten en netwerkisolatie afgedwongen?** De PoC gebruikt Docker runtime-opties voor CPU, geheugen en PID-limieten en maakt per runtime-instantie een eigen bridge-netwerk aan.
- **Hoe communiceert CTFd met Docker?** In deze bachelorproef gebeurt dat via een eigen plugin in Python die de Docker SDK voor Python gebruikt om containers, netwerken en image-imports aan te sturen. Die communicatie

werkt zowel tegen een lokale Docker-socket als tegen een remote Docker API op een andere host.

- **Hoe werd de stabiliteit gevalideerd?** Via een combinatie van geautomatiseerde tests, smoke tests, timeout-validatie, isolatie-experimenten, een beperkte loadtest met Locust en split-host validatie in zowel een lokale simulatie als een reële twee-VM-opstelling.

8.2. Beperkingen van de PoC

Hoewel de plugin-PoC de kern van de onderzoeksvraag beantwoordt, zijn er duidelijke beperkingen. De huidige implementatie werd wel gevalideerd in same-host modus en in een split deployment over twee VM's, maar ze toont nog niet hoe de oplossing zich gedraagt in een bredere meerhost-opstelling met taakverdeling over meerdere runtime-hosts. Ook reverse-proxy-routing, TLS-beveiligde remote Docker-communicatie, geavanceerde production hardening en aangepaste seccomp-profielen maken nog geen afgerond deel uit van de PoC.

Daarnaast was de standalone PoC nuttig als exploratief tussenresultaat, maar ze is niet bedoeld als eindarchitectuur. De standalone stap diende vooral om risico's vroeg te reduceren en technische keuzes te onderbouwen voordat de pluginintegratie werd gebouwd.

8.3. Aanbevelingen voor productiegebruik

Voor productiegebruik volstaat de huidige PoC nog niet zonder bijkomende maatregelen. Organisaties die verder willen bouwen op deze aanpak, moeten minstens rekening houden met:

- hardening van de Docker-host en het netwerksegment waarin challenge-containers draaien;
- versleutelde en streng afgeschermdde communicatie indien Docker niet lokaal op dezelfde host draait;
- een expliciet beheer van gepubliceerde poortbereiken en firewallregels op de runtime-host;
- bijkomende monitoring en alerting voor mislukte starts, cleanupfouten en capaciteitspieken;
- operationele procedures voor imagebeheer, updates en incidentrespons.

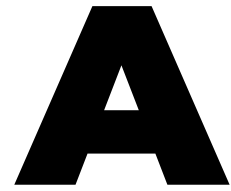
De plugin-aanpak blijft daarbij een passende basis, omdat ze de gebruikerservaring en het beheer in CTFd centraliseert en tegelijk voldoende controle biedt over runtime-beleid.

8.4. Mogelijke uitbreidingen

De belangrijkste volgende stap is het hardenen van de reeds bewezen split-host architectuur. Daarbij moeten minstens vier vragen beantwoord worden:

- hoe remote Docker-communicatie via TLS en passende toegangscontrole concreet wordt opgezet;
- of een dunne tussenlaag toch wenselijk is tussen plugin en Docker-host voor extra beveiliging of foutafhandeling;
- wat de impact is van die opsplitsing op opstartlatency en cleanupbetrouwbaarheid onder hogere belasting;
- hoe het systeem reageert wanneer de Docker-host tijdelijk onbereikbaar is.

Andere interessante uitbreidingen zijn een reverse-proxy-gebaseerd toegangspunt, uitgebreidere loadtesting en verdere production hardening.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

Samenvatting

Dit onderzoeksvoorstel behandelt het dynamisch beheer van Docker-containers voor Capture The Flag (CTF)-events op on-premise infrastructuur. De probleemstelling richt zich op het aanmaken en beëindigen van containers op basis van gebruikersacties. De hoofdonderzoeksvraag onderzoekt hoe dit proces kan worden geautomatiseerd met behulp van CTFd en Docker. Daarbij wordt gewerkt binnen de randvoorwaarden van een volledig self-hosted omgeving, zonder terugkerende licentiekosten en met een voorkeur voor open-source componenten. Via een literatuurstudie en een proof-of-concept worden de beveiligingsaspecten, resource-limieten en communicatie tussen CTFd en Docker onderzocht. Het doel is een schaalbare oplossing te ontwerpen die misbruik helpt voorkomen en de prestatie bewaakt.

A.1. Introductie

Dit onderzoeksvoorstel richt zich op het dynamisch beheer van Docker-omgevingen voor Capture The Flag (CTF) events (Innovirtuoso, [z.d.](#)) op on-premise infrastructuur. Het idee voor dit onderzoek komt voort uit de noodzaak om challenges efficiënt en veilig te beheren (Singier, [2025](#)).

Kader bij de uiteindelijke uitwerking: tijdens de realisatie van de bachelorproef werd eerst een beperkte standalone proefopstelling gebouwd om technische haalbaarheid te toetsen. Die stap diende als verkennende tussenfase; de uiteindelijke aanbeveling bleef een plugin-aanpak binnen CTFd.

A.1.1. Probleemstelling

De centrale probleemstelling luidt: Bestaat er een manier om dynamisch Docker-omgevingbeheer te doen op on-prem infrastructuur tijdens Capture The Flag events? Het beheren van containers voor challenges vereist een robuust systeem dat kan omgaan met gebruikersacties en variabele belastingen, zonder in te boeten op veiligheid of performance.

Daarnaast is het een vereiste dat de oplossing volledig self-hosted kan worden ingezet. Hiervoor zijn drie belangrijke redenen: flexibiliteit (mogelijkheid tot uitbreidingen en features), kostprijs (het betalen van on-premise infrastructuur versus cloud-diensten is voor veel organisaties een andere financiële flow) en de voorkeur voor open-source oplossingen.

A.1.2. Onderzoeksvraag

De hoofdonderzoeksvraag van dit bachelorproefvoorstel is:

Hoe kunnen we dynamisch onze docker containers gelinkt aan CTFd-challenges beheren, aanmaken en beëindigen op basis van gebruikersacties en ingestelde variabelen?

Om deze hoofdvraag te beantwoorden, worden de volgende deelvragen geformuleerd, opgesplitst in probleem- en oplossingsdomein:

Probleemdomein:

- Welke fundamentele en direct toepasbare beveiligingsmaatregelen ('low-hanging fruit') zijn noodzakelijk om de veiligheid van de Docker-host en -containers te waarborgen?
- Welke specifieke beveiligingsrisico's brengt het dynamisch aanmaken van containers door gebruikers met zich mee op on-premise infrastructuur?
- Wat is de impact van frequente container-creatie en -vernietiging op de performance van de host-server?
- Welke communicatieprotocollen zijn geschikt voor de interactie tussen het CTF-platform en de container-runtime?

Oplossingsdomein (gericht op de Proof-of-Concept):

- Hoe kan een CTFd-plugin ontwikkeld worden die rechtstreeks communiceert met de Docker Engine API zonder zware externe dependencies?
- Op welke manier kunnen resource-limieten (CPU, RAM) en netwerkisolatie technisch worden afgedwongen in de PoC?
- Hoe kan de architectuur zo worden opgezet dat deze volledig self-hosted en modulair uitbreidbaar is?

- Hoe valideren we de stabiliteit van de oplossing onder gesimuleerde zware belasting (load testing)?

A.1.3. Onderzoeksdoelstelling

Het doel is om een werkende oplossing of proof-of-concept te realiseren die aan toont hoe CTFd-challenges dynamisch en veilig beheerd kunnen worden op eigen infrastructuur.

A.2. State-of-the-art

Er is al documentatie beschikbaar over het deployen van challenges binnen CTFd (CTFd, [z.d.](#)). Echter, het veilig en dynamisch beheren van deze containers op eigen infrastructuur brengt specifieke uitdagingen met zich mee.

Beveiliging is een cruciaal aspect; containers moeten geïsoleerd zijn om te voorkomen dat deelnemers dingen kunnen doen op de host of andere containers. Bronnen zoals de Docker Security Cheat Sheet van OWASP (OWASP, [2024](#)) en artikelen over container security (Aikido, [2025](#)) bieden richtlijnen hiervoor.

Daarnaast biedt een overzicht van het Docker-ecosysteem inzicht in de beschikbare tools en technologieën die ingezet kunnen worden (Muzumdar e.a., [2024](#)). Het onderzoek zal zich baseren op deze bronnen om een veilige en schaalbare architectuur te ontwerpen.

A.3. Methodologie

Het onderzoek zal bestaan uit een combinatie van literatuurstudie en praktijkgericht onderzoek.

In de eerste fase wordt via literatuurstudie onderzocht welke 'low-hanging fruit' beveiligingsmaatregelen essentieel zijn. De focus ligt hierbij op praktische en effectieve best practices voor container security (zoals resource limits, netwerkisolatie en privilege management) in plaats van diepgaande theoretische beveiligingsmodellen. Daarnaast wordt gekeken naar de Docker Engine API en SDKs voor de integratie.

Vervolgens zal een proof-of-concept (PoC) worden ontwikkeld waarin een CTFd-omgeving wordt opgezet die communiceert met een Docker-daemon. Hierbij wordt gefocust op de implementatie van een API-interface, het instellen van resource limieten en het toepassen van de geïdentificeerde basisbeveiliging.

Tot slot zal de oplossing gevalideerd worden aan de hand van een uitgebreid testplan als onderdeel van de PoC-verificatie. Dit testplan beschrijft de strategie om zowel de stabiliteit als de veiligheid van het systeem te waarborgen. Concreet worden er load tests uitgevoerd om de schaalbaarheid en performance onder zware belasting te meten. Daarnaast worden specifieke beveiligingstests (zoals container escape pogingen en resource exhaustion simulaties) uitgevoerd om te verifiëren of

de geïmplementeerde isolatie en restricties effectief zijn tegen misbruik.

A.4. Verwacht resultaat, conclusie

Het verwachte resultaat is een gedocumenteerde en werkende implementatie voor dynamisch containerbeheer binnen CTFd. Dit omvat:

- Een adviesrapport over de te gebruiken architectuur en beveiligingsmaatregelen.
- Een proof-of-concept implementatie.
- Testresultaten die de schaalbaarheid en stabiliteit aantonen.
- Installatie- en beheerinstructies om de reproduceerbaarheid van de PoC te waarborgen.
- Technische documentatie van de ontwikkelde architectuur, zodat derden hierop kunnen verder bouwen.

De meerwaarde van dit onderzoek ligt in het vereenvoudigen van het beheer van CTF-events op eigen hardware, met behoud van veiligheid en performance.

B

Installatie- en beheerinstructies

Deze bijlage beschrijft hoe beide PoC's gereproduceerd kunnen worden. De standalone PoC is opgenomen als voorstudie; de plugin-PoC is het hoofdresultaat van de bachelorproef.

Standalone PoC

De standalone PoC kan gebruikt worden om de kernmechanismen los van CTFd te reproduceren.

1. Zorg dat Docker Desktop of Docker Engine actief is en dat Docker Compose v2 beschikbaar is.
2. Navigeer naar de map POC/.
3. Start de end-to-end smoke test via `./scripts/smoke_test.sh`.
4. Open nadien het dashboard op `http://127.0.0.1:8000`.

Voor lokale tests zijn Python 3.12 en de dependencies uit `app/requirements-dev.txt` nodig. De tests kunnen uitgevoerd worden met `app/.venv/bin/pytest -q tests`.

CTFd-plugin PoC

De plugin-PoC is de primaire reproductieroute voor deze bachelorproef.

1. Zorg dat Docker actief is en dat Docker Compose v2 beschikbaar is.
2. Navigeer naar de map POC-CTFd/.
3. Start de same-host validatie via `./scripts/smoke_test.sh`.

4. Start de split-host validatie via `./scripts/smoke_test_split.sh` wanneer de control plane en runtime-host logisch gescheiden moeten worden.
5. Open CTFd op `http://127.0.0.1:8001`.

De same-host smoke test initialiseert CTFd, uploadt een Docker-archief, maakt een containerized challenge aan, start runtime-instanties voor meerdere spelers, controleert bereikbaarheid en valideert cleanup na solve, manuele stop en timeout. De split-host smoke test doorloopt dezelfde functionele flow, maar laat CTFd communiceren met een afzonderlijke Docker-runtime.

Een reële twee-VM reproductie is bijkomend gedocumenteerd in `P0C-CTFd/docs/03_real-vm-validation.md`. Daarin wordt een Lima-opstelling beschreven met CTFd en plugin op een eerste VM en de challenge-containers op een tweede VM.

Beheer

Voor de plugin-PoC gebeurt beheer grotendeels vanuit CTFd zelf. De plugin voegt een administratieve runtime-pagina toe waarop actieve instanties zichtbaar zijn. Daar kan een beheerder:

- individuele instanties geforceerd stoppen;
- alle actieve instanties van een challenge tegelijk stopzetten;
- de timeout-reaper manueel uitvoeren;
- per challenge het resourceprofiel raadplegen.

Voor de standalone PoC bestaan daarnaast helper-scripts, zoals `cleanup_runtime.sh`, om alle beheerde runtime-artifacten op te ruimen.



Configuratiebestanden

Deze bijlage vat de belangrijkste configuratie-elementen samen die voor de PoC relevant zijn.

Plugin-configuratie

De plugin voegt per challenge de volgende configuratievelden toe:

- image of geupload Docker-archief;
- een of meerdere containerpoorten;
- CPU-limiet;
- geheugenlimiet;
- timeout in seconden;
- maximum aantal gelijktijdige instanties.

Daarnaast ondersteunt de plugin optioneel een geupload Docker-archief dat lokaal wordt opgeslagen en nadien in de Docker-daemon geïmporteerd of opnieuw geïmporteerd kan worden.

Runtime-configuratie via environment variables

De plugin gebruikt onder meer environment variables voor:

- CTFD_CONTAINER_DOCKER_HOST voor het Docker-control endpoint;
- CTFD_CONTAINER_DOCKER_TLS_VERIFY voor TLS-verificatie richting Docker;
- CTFD_CONTAINER_DOCKER_CERT_PATH voor clientcertificaten richting Docker;

- CTFD_CONTAINER_PUBLIC_HOST voor spelergerichte runtime-URL's;
- CTFD_CONTAINER_PUBLIC_SCHEME voor het URL-schema van die runtime-URL's;
- CTFD_CONTAINER_PUBLISHED_PORT_MIN en CTFD_CONTAINER_PUBLISHED_PORT_MAX voor een optioneel begreemd poortbereik;
- CTFD_CONTAINER_DB_PATH voor de runtime-database;
- CTFD_CONTAINER_IMAGE_ARCHIVE_DIR voor opslag van geuploadede image-archieven;
- CTFD_CONTAINER_REAPER_INTERVAL voor het interval van de timeout-reaper.

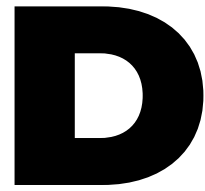
Container hardening

Bij het starten van een challenge-container worden standaard de volgende runtime-opties toegepast:

- read-only root filesystem;
- `cap_drop=[ 'ALL' ]`;
- `security_opt=[ 'no-new-privileges:true' ]`;
- tmpfs voor /tmp;
- CPU-, geheugen- en PID-limieten;
- een afzonderlijk bridge-netwerk per instantie.

Niet-opgenomen configuratie

Reverse-proxy-configuratie en taakverdeling over meerdere runtime-hosts zijn bewust niet opgenomen als afgewerkte configuratiebestanden. De split-host opstelling met één aparte runtime-host werd wel gevalideerd, maar productierijpe TLS-configuratie en meerhost-orkestratie blijven toekomstig werk.



Testresultaten in detail

Deze bijlage bundelt de belangrijkste commando's en samengevatte resultaten die in de validatiehoofdstukken gebruikt werden.

Standalone PoC

Pytest

```
cd POC
app/.venv/bin/pytest -q tests
```

Resultaat: 4 passed in 0.60s

Smoke test

```
cd POC
./scripts/smoke_test.sh
```

Resultaat: de demo-image en orchestrator werden opgebouwd, twee instanties werden gestart, de HTTP-endpoints waren bereikbaar en beide instanties werden nadien correct gestopt.

Load snapshot

```
cd POC
app/.venv/bin/locust \
  -f tests/locustfile.py \
  --host=http://127.0.0.1:8000 \
  --headless -u 10 -r 5 -t 10s
```

Samenvatting: 128 requests, 84 failures, telkens door 409 CONFLICT na het bereiken van max_instances.

Plugin-PoC

End-to-end smoke test

```
cd POC-CTFd
./scripts/smoke_test.sh
```

Resultaat: CTFd werd geïntialiseerd, een Docker-archief werd geupload, een containerized challenge werd aangemaakt, de lokaal geïmporteerde image werd verwijderd om herimport te forceren, meerdere spelers startten runtimes, max_instances werd afgedwongen, een tweede start hergebruikte dezelfde instantie, manuele stop en timeout cleanup werden bevestigd en na solve werd de runtime automatisch opgeruimd met stopreden challenge-solved.

Split-host smoke test

```
cd POC-CTFd
./scripts/smoke_test_split.sh
```

Resultaat: dezelfde challenge-flow werkte ook wanneer CTFd met de plugin via een remote Docker endpoint communiceerde. De gegenereerde runtime-URL's verwezen naar de afzonderlijke runtime-host en de gepubliceerde poorten bleven binnen het ingestelde bereik.

Reële twee-VM validatie

De reële validatie gebruikte twee Lima-VM's op een gedeeld user-v2-netwerk. De eerste VM draaide CTFd en de plugin; de tweede VM draaide de Docker Engine voor challenge-containers. De belangrijkste setupstappen waren:

```
limactl network create bp-shared-net --gateway 192.168.150.1/24
```

```
limactl start --name=bp-ctfd-vm \
  --network=lima:bp-shared-net \
  --cpus=2 --memory=4 --disk=20 \
  --mount=/Users/manu/Documents/GitHub/Bachelorproef:w \
  template:docker-rootful -y
```

```
limactl start --name=bp-docker-vm \
  --network=lima:bp-shared-net \
  --cpus=2 --memory=4 --disk=20 \
  template:docker-rootful -y
```

Op de runtime-VM werd de Docker-daemon voor deze wegwerpvalidatie ook op TCP-poort 2375 beschikbaar gemaakt. Dat gebeurde enkel om remote Docker-aansturing te testen; voor productie blijft TLS-beveiliging vereist.

```
ExecStart=/usr/bin/dockerd \
```

```
--host=unix:///var/run/docker.sock \  
--host=tcp://0.0.0.0:2375 \  
--containerd=/run/containerd/containerd.sock
```

Daarna werd de CTFd-stack op de control-VM gestart met de runtime-VM als Docker-control endpoint en als publieke challenge-host:

```
export CTFD_CONTAINER_DOCKER_HOST=tcp://192.168.150.3:2375  
export CTFD_CONTAINER_PUBLIC_HOST=192.168.150.3  
export CTFD_CONTAINER_PUBLISHED_PORT_MIN=20000  
export CTFD_CONTAINER_PUBLISHED_PORT_MAX=20099  
docker compose -f docker-compose.yml up -d --build
```

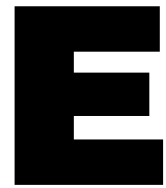
De smoke test werd vanuit de CTFd-VM uitgevoerd, omdat de macOS-host het interne Lima-subnet in deze netwerkmodus niet rechtstreeks kon bereiken:

```
BASE_URL=http://127.0.0.1:8001 \  
IMAGE_ARCHIVE_PATH="$archive_path" \  
SMOKE_TEST_DOCKER_HOST=tcp://192.168.150.3:2375 \  
SMOKE_TEST_EXPECT_ACCESS_HOST=192.168.150.3 \  
SMOKE_TEST_EXPECT_ACCESS_PORT_MIN=20000 \  
SMOKE_TEST_EXPECT_ACCESS_PORT_MAX=20099 \  
python3 ./scripts/smoke_test.py
```

Resultaat: op 24 maart 2026 werd bevestigd dat CTFd op een eerste Lima-VM challenge-containers kon starten, stoppen en opruimen op een tweede Lima-VM. De smoke test rapporteerde `Smoke test passed`. Na afloop bevestigden `docker ps -a` en `docker network ls` op de runtime-VM dat geen plugin-beheerde containers of netwerken waren achtergebleven. De volledige log is opgenomen in `POC-CTFd/docs/03_real-vm-validation.md`.

Interpretatie

De gedetailleerde outputs tonen vooral aan dat de gekozen lifecycle-regels effectief worden afgedwongen. Ze tonen ook aan dat split deployment technisch haalbaar is met dezelfde pluginarchitectuur. Ze vormen wel nog geen bewijs voor een volledig afgewerkte productiearchitectuur met TLS-beveiligde remote Docker-communicatie, reverse proxy en taakverdeling over meerdere runtime-hosts.



Onderzoekslog

Dit appendix documenteert de belangrijkste beslissingen die tijdens het onderzoek werden genomen, met hun motivatie. Het geeft inzicht in het verloop van het onderzoeksproces en de keuzes die de uiteindelijke proof-of-concept hebben gevormd.

Fase 1 – Literatuurstudie en begrippenkader

Beslissing: afbakening van de literatuurstudie

Bij de start van het onderzoek werd beslist de literatuurstudie te richten op vier thema's: CTF-events en het CTFd-platform, Docker en containerisatie, container-beveiliging en load balancing zonder Kubernetes. Dit stelde de lezer in staat de rest van de bachelorproef te volgen zonder voorkennis. Diepgaande theoretische beveiligingsmodellen (zoals formele verificatie of CVE-analyse) werden bewust buiten scope gelaten ten voordele van direct toepasbare best practices (Center for Internet Security, [2023](#); OWASP, [2024](#)).

Beslissing: focus op “low-hanging fruit” beveiliging

Na het bestuderen van de OWASP Docker Security Cheat Sheet (OWASP, [2024](#)) en het CIS Docker Benchmark (Center for Internet Security, [2023](#)) werd gekozen om de focus te leggen op praktisch toepasbare basismaatregelen: capabilities droppen, read-only bestandssysteem, seccomp-profielen, no-new-privileges en resource-limieten. Deze maatregelen bieden een goede balans tussen beveiliging en implementatiecomplexiteit voor een CTF-context.

Fase 2 – Analyse van de huidige en gewenste situatie

Beslissing: Kubernetes niet als primaire oplossing

Kubernetes werd vroeg in het onderzoek overwogen als orkestratieplatform. Na analyse werd besloten het niet als primaire oplossing te gebruiken omdat de opera-

tionele complexiteit (clusteropzet, beheer, leercurve) niet in verhouding staat met de schaal van een doorsnee CTF-event. Als alternatief werd gefocust op Docker Swarm (Docker Inc., [z.d.-d](#)) en Traefik (Traefik Labs, [z.d.](#)) als lichtere alternatieven. Dit sluit ook aan bij de randvoorwaarde van eenvoudige inzetbaarheid op eigen infrastructuur.

Beslissing: granulariteit – container per gebruiker of per team

Bij de architectuuranalyse werd de keuze tussen een instantie per gebruiker en een instantie per team bestudeerd. Per gebruiker biedt maximale isolatie maar verhoogt het aantal containers snel. Per team beperkt het resource-gebruik maar verlaagt de isolatiegarantie. Uiteindelijk werd gekozen voor een flexibele aanpak: in teammodus wordt één instantie per team voorzien, bij individuele deelname één per gebruiker. Dit koppelt de granulariteit aan het wedstrijdmodel en vermijdt onnodige containerexplosie.

Beslissing: Docker SDK voor Python vs. directe API-calls

Voor de integratie tussen de CTFd-plugin en de Docker-daemon werden twee opties afgewogen: de officiële Docker SDK voor Python (Docker Inc., [z.d.-c](#)) en directe HTTP-aanroepen naar de Docker Engine API (Docker Inc., [z.d.-b](#)). De SDK biedt een hogere abstractie, minder foutgevoeligheid en is de aanbevolen aanpak voor Python-applicaties. Directe API-calls zouden meer controle geven maar ook meer boilerplate en foutafhandeling vergen. Gekozen werd voor de SDK, tenzij er specifieke beperkingen opduiken tijdens de implementatie.

Beslissing: reverse proxy niet in de eerste implementatie

Een reverse proxy met dynamische routing, bijvoorbeeld via Traefik (Traefik Labs, [z.d.](#)), bleef architecturaal interessant, maar werd niet in de eerste implementatiefase opgenomen. De reden daarvoor was pragmatisch: eerst moest duidelijk worden dat container lifecycle management, cleanup en basisbeveiliging betrouwbaar werkten. Reverse-proxy-routing werd daarom verschoven naar een latere uitbreidingsfase.

Beslissing: eerst een verkennende standalone stap, daarna plugin-integratie

Hoewel de einddoelstelling altijd plugin-gebaseerd bleef, werd tijdens de analyse beslist eerst een beperkte standalone PoC te bouwen. Die keuze liet toe om de kern van container lifecycle management los van CTFd te valideren en technische risico's vroeg te reduceren. Zodra die basis stabiel genoeg bleek, werd de focus verschoven naar de eigenlijke CTFd-plugin.

Beslissing: bestaande plugin CTFd-Whale als referentie, niet als basis

De bestaande CTFd-Whale plugin (Glzjin, [z.d.](#)) werd bestudeerd als referentie voor de architectuuranalyse. Er werd besloten geen fork of uitbreiding te maken, maar een eigen implementatie te schrijven. CTFd-Whale voldoet niet volledig aan de randvoorwaarden van dit onderzoek (volledig self-hosted, open-source, controleerbare beveiligingsinstellingen) en een eigen implementatie biedt meer controle over de beveiligings- en resourceconfiguratie.

Fase 3 – Uitwerking en heroriëntatie

Beslissing: standalone PoC behouden als voorstudie, niet als eindarchitectuur

Na de eerste experimenten bleek dat de standalone PoC bruikbaar was voor het valideren van timeouts, netwerkisolatie, logging en basisresourcebeleid. Tegelijk werd duidelijk dat deze opzet minder goed aansloot bij de centrale onderzoeksvraag, omdat de gebruikersinteractie dan buiten CTFd bleef. De standalone PoC werd daarom behouden als onderzoeksstap en niet als finale oplossingsrichting.

Beslissing: plugin behouden als voorkeursrichting

Na de realisatie van de plugin-PoC werd beslist om de plugin als voorkeursrichting naar voren te schuiven. De plugin koppelt gebruikersacties rechtstreeks aan challenge-runtimes, houdt de beheerinterface binnen CTFd en sluit daardoor goed aan bij de context van CTF-events op eigen infrastructuur.

Fase 4 – Validatie en afronding

Beslissing: split deployment als aparte validatiestap uitvoeren

Tijdens de eerste uitwerking werd split deployment nog niet meteen als resultaat gepresenteerd. Er werd beslist om dit niet speculatief in de PoC-hoofdstukken op te nemen, maar pas na een afzonderlijke validatie. In een latere iteratie werd daarvoor zowel een lokale split-host simulatie als een reële twee-VM test op Lima uitgevoerd. Op die manier kon de haalbaarheid van de architectuur alsnog onderbouwd worden met concrete meet- en testresultaten.

Beslissing: remote Docker-communicatie aantonen, maar niet als production-ready voorstellen

De extra validatie toonde aan dat de plugin een remote Docker-host kan aansturen en correcte spelergerichte runtime-URL's kan genereren. Tegelijk werd beslist dit niet als volledig productieklare oplossing te presenteren, omdat de gebruikte testopstellingen nog geen TLS-beveiligde remote API, uitgebreide foutinjectie of taakverdeling over meerdere runtime-hosts omvatten.

Bibliografie

- Aikido. (2025, april 28). *Docker & Kubernetes Container Security Explained*. Verkregen november 28, 2025, van <https://www.aikido.dev/blog/docker-kubernetes-container-security>
- Center for Internet Security. (2023). *CIS Docker Benchmark* (tech. rap.). Center for Internet Security. Verkregen februari 22, 2026, van <https://www.cisecurity.org/benchmark/docker>
- CTFd. (z.d.). *Deploying Challenge Services*. Verkregen november 28, 2025, van <https://docs.ctfd.io/tutorials/challenges/deploying-challenges>
- CTFd LLC. (z.d.-a). *Configuration*. Verkregen mei 20, 2026, van <https://docs.ctfd.io/docs/deployment/configuration/>
- CTFd LLC. (z.d.-b). *Contribution Guidelines*. Verkregen mei 20, 2026, van <https://docs.ctfd.io/docs/contributing/overview/>
- CTFd LLC. (z.d.-c). *CTFd: CTFs as you need them*. Verkregen februari 22, 2026, van <https://github.com/CTFd/CTFd>
- CTFd LLC. (z.d.-d). *Getting Started with the CTFd REST API*. Verkregen mei 20, 2026, van <https://docs.ctfd.io/docs/api/getting-started/>
- CTFd LLC. (z.d.-e). *Installation*. Verkregen mei 20, 2026, van <https://docs.ctfd.io/docs/deployment/installation/>
- CTFd LLC. (z.d.-f). *Overview*. Verkregen mei 20, 2026, van <https://docs.ctfd.io/docs/overview/>
- CTFd LLC. (z.d.-g). *Plugins Overview*. Verkregen mei 20, 2026, van <https://docs.ctfd.io/docs/plugins/overview/>
- Docker Inc. (z.d.-a). *Docker Compose overview*. Verkregen februari 22, 2026, van <https://docs.docker.com/compose/>
- Docker Inc. (z.d.-b). *Docker Engine API*. Verkregen februari 22, 2026, van <https://docs.docker.com/engine/api/>
- Docker Inc. (z.d.-c). *Docker SDK for Python*. Verkregen februari 22, 2026, van <https://docker-py.readthedocs.io/>
- Docker Inc. (z.d.-d). *Swarm mode overview*. Verkregen februari 22, 2026, van <https://docs.docker.com/engine/swarm/>
- Docker Inc. (z.d.-e). *What is Docker?* Verkregen mei 20, 2026, van <https://docs.docker.com/get-started/docker-overview/>

- Glzjin. (z.d.). *CTFd-Whale: CTFd Plugin for Deploying Standalone Challenges as Docker Containers*. Verkregen februari 22, 2026, van <https://github.com/glzjin/CTFd-Whale>
- Google. (z.d.). *kCTF: Kubernetes-based CTF hosting*. Verkregen februari 22, 2026, van <https://github.com/google/kctf>
- HAProxy Technologies. (z.d.). *HAProxy: The Reliable, High Performance TCP/HTTP Load Balancer*. Verkregen februari 22, 2026, van <https://www.haproxy.org/>
- Innovirtuoso. (z.d.). *Capture The Flag (CTF) Explained: How Hacking Competitions Turn You Into a Cyber Pro*. Verkregen december 11, 2025, van <https://innovirtuoso.com/cybersecurity/capture-the-flag-ctf-explained-how-hacking-competitions-turn-you-into-a-cyber-pro/>
- Locust contributors. (z.d.). *Locust: An open source load testing tool*. Verkregen februari 22, 2026, van <https://locust.io/>
- Muzumdar, P., Bhosale, A., Basyal, G. P., & Kurian, G. (2024). Navigating the Docker Ecosystem: A Comprehensive Taxonomy and Survey. <https://arxiv.org/pdf/2403.17940.pdf>
- OWASP. (2024). *Docker Security Cheat Sheet*. Verkregen november 28, 2025, van https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- redpwn. (z.d.). *rCTF: redpwn's CTF platform*. Verkregen februari 22, 2026, van <https://github.com/redpwn/rctf>
- Singier, L. (2025). Ideeendocument BP.
- The Kubernetes Authors. (z.d.). *Kubernetes Overview*. Verkregen mei 20, 2026, van <https://kubernetes.io/docs/concepts/overview/>
- Traefik Labs. (z.d.). *Traefik Proxy Documentation*. Verkregen februari 22, 2026, van <https://doc.traefik.io/traefik/>